arXiv:2606.06310v3 [cs.CG] 11 Sep 2026

RedZeD: Computing persistent homology by Reduction to Zero Differentials

Krzysztof Kapulkin Nathan Kershaw

September 14, 2026

Abstract

We introduce a new algorithm for computing persistent homology of Vietoris–Rips filtrations, which offers a considerable improvement both in terms of time and memory over the existing implementations of the persistence pairing algorithm. The key innovation, called active enumeration, is made possible by a new theoretical framework of Reduction to Zero Differentials (hence RedZeD) in which to view persistent homology.

Keywords: topological data analysis, persistent homology, filtration, chain complex, reduction to zero differentials

Introduction

Persistent homology is a fundamental tool of topological data analysis, and, more broadly, of computational topology [ZC05, ELZ02, OPT$^+$17]. It takes as input a filtration (or often a more general diagram) of chain complexes and returns a persistence barcode which tracks topological features of that input filtration. Examples of filtrations of chain complexes abound in both pure and applied mathematics, arising from (filtrations of) topological spaces, simplicial and cubical complexes, simplicial and cubical sets, graphs, and data. Consequently, considerable effort has been put into designing algorithms for efficiently computing this invariant [CK11, dSMVJ11, BKR14, BDM15, BKRW17, OPT$^+$17, Bau21, BMG$^+$24].

The Vietoris–Rips filtrations arising from finite metric spaces, or more generally sets equipped with a distance function (i.e., a function $d\colon X \times X \to \mathbb{R}_+$ that is symmetric and satisfies $d(x,x) = 0$), are of particular importance in topological data analysis and topological inference. Given a finite metric space, its Vietoris–Rips filtration is a filtration of simplicial complexes with vertex set X, and with higher simplices at scale $r \geq 0$ given by finite subsets of points that are pairwise at most r apart. The persistent homology of this filtration can be used to detect topological features of the input that persist through different time scales.

The *persistence pairing* algorithm [ZC05], also referred to as the *standard* algorithm, is the basic tool for computing persistent homology of a filtration of chain complexes. It uses restricted Gaussian elimination, reducing columns left-to-right by adding previously reduced columns to either clear a column or identify a unique pivot. Since its introduction, many improvements to the algorithm have been proposed. Broadly speaking, these improvements fall into two categories: reducing the size of chain complexes involved in the filtration at hand, or providing more efficient tools for generating and reducing the matrix. Unsurprisingly, amongst the broad array of these tools, there is significant variance when it comes to the generality they apply in and how much speedup they offer.

In the specific case of the Vietoris–Rips filtration mentioned above, the software Ripser [Bau21] has emerged as the gold standard for efficient computation of persistent homology. By combining different improvements such as: clearing birth columns [CK11], computing cohomology instead of homology

[dSMVJ11], implicit matrix reductions, and identifying apparent and emerging pairs, Ripser outperforms other implementations both in terms of computational speed and memory usage.

The purpose of the present work is twofold. First, we provide an abstract rephrasing and a generalization to the level of chain complexes of some of the existing algorithms. Second, we introduce a new speedup that is specific to persistent homology of Vietoris–Rips complexes.

The first of these goals is accomplished by placing the persistence pairing algorithm in a more abstract context. Specifically, we show that it can be seen as replacing a given filtration of chain complexes (over a field) by a naturally quasi-isomorphic sequence of chain complexes with zero differentials. Let us unpack this statement. If a chain complex has zero differentials, its homology groups are simply the chain groups themselves. Thus, given a filtration of chain complexes C^*, we might wish to replace it with a new sequence of chain complexes A^* (which might no longer be a filtration of subcomplexes) with zero differentials, along with a family of quasi-isomorphisms $r^*\colon C^* \to A^*$, i.e., maps inducing isomorphisms on homology. The key insight is that each chain complex A^i in the sequence A^* depends only on the previous chain complex A^{i-1} and the map $r^{i-1}\colon C^{i-1} \to A^{i-1}$. At its core, this observation is not new and was made in the context of cochain complexes arising from simplicial complexes in [DFW14], where the map r mentioned above is called an *annotation*. The novel contribution of our work up to this point is a more abstract treatment of this insight.

Algorithmically, at each step of the filtration, we can store r using dictionaries R_0, R_1, …, R_n corresponding to the homological degree. With this, a straightforward implementation of the proof of the aforementioned fact leads to the first algorithm considered in the present paper, named RedZeD, which is an acronym for **Red**uction to **Ze**ro **D**ifferentials. While seemingly approaching the problem from a different direction, RedZeD can be rephrased in the language of matrix reduction algorithms, in which case it can be seen to implement several previously studied improvements to the standard algorithm, namely, *compression* [BKR14] along with *exhaustive and retrospective reduction* [BMG+24, EO20, EZ03, ZC05]. In other words, RedZeD represents both a generalization and a more abstract treatment, along with a new implementation of several existing ideas.

On its own, RedZeD does not offer significant improvements over existing methods. This new perspective, however, suggests a new improvement to the algorithm specific to the Vietoris–Rips filtration. This improvement is only possible on top of RedZeD, making it a necessary component. Typically, when computing persistent homology of Vietoris–Rips complexes up to degree n, the majority of time is spent reducing $(n+1)$-birth columns, i.e., columns creating new features in degree $n+1$, but not affecting degree n. Consequently, one wishes to avoid considering such $(n+1)$-simplices altogether, or at the very least "weed out" as many of them as possible. In Ripser, this is done by computing cohomology and birth clearing, neither of which apply to our homological setting. Instead, we employ a technique which we refer to as *active enumeration*. We say that a birth simplex σ is *active* at time i if $r^i(\sigma) \neq 0$, and it is *inactive* otherwise. This cuts across the standard distinction between alive and dead simplices used in persistence, as now a simplex can be simultaneously both dead and active.

After adding a new birth n-simplex σ, active enumeration searches for all $(n+1)$-simplices τ that have at least one active face and σ as their youngest face. This can be done by an efficient check: loop through all active n-simplices, check whether they share an $(n-1)$-face with σ, then check whether a single vertex can be added to σ to form a desired $(n+1)$-simplex. The key point is that RedZeD's use of dictionaries $R_0, R_1, \ldots, R_n$ makes it immediate to identify active n-simplices — they are exactly the keys of R_n. This constitutes RedZeD_VR, a specialization of RedZeD to the Vietoris–Rips filtration.

To ascertain the quality of improvements implemented in RedZeD_VR, relative to other software, notably Ripser, we run several tests comparing their performance across different data sets. These include both curated data sets with known characteristics, and common benchmark data sets found in [Bau21]. We test 22 different cases, and find that RedZeD_VR is faster in 21 of them, the one outlier being H_3 of a noisy 3-dimensional sphere. In many cases, RedZeD_VR also scales better than Ripser. These results are most pronounced for H_1, especially on data with more topological signal or high-dimensional data. Speedups in higher homology degrees are not as pronounced as in degree 1, but RedZeD_VR generally

remains faster, aside from the one aforementioned case. We also find that RedZeD_VR generally uses less peak memory than Ripser. Again, these results are most pronounced in degree 1, where RedZeD_VR always uses less memory. RedZeD_VR generally uses less memory in higher degrees as well; however, Ripser tends to outperform RedZeD_VR in higher degrees for data with little topological signal, or for large datasets computed with a low threshold. Some of the more notable results for H_1 can be seen in the table below:

			Time (s)			Peak memory (MiB)		
Dataset		pts	RedZeD_VR	Ripser	Speedup	RedZeD_VR	Ripser	Ratio
grid spheres, S^1	H_1	5,000	0.706	171	242	167	452	2.71
noisy sphere, S^1	H_1	3,800	3.87	292	75.4	231	16,500	71.4
random distance	H_1	450	0.404	251	621	30.9	2,070	66.9
random euclidean, $\mathbb{R}^{10}$	H_1	6,000	1.12	86.5	76.9	258	4,400	17.1
`o3_1024`	H_1	1,024	0.0599	0.898	15.0	16.4	145	8.82
`dragon`	H_1	2,000	0.114	0.969	8.51	30.0	98.4	3.28
`o3_4096`	H_1	4,096	1.40	53.3	38.0	177	8,360	47.2

Table 1: Time and memory comparisons between RedZeD_VR and Ripser for various datasets

Organization of the paper

The paper is organized into six sections. Section 1 treats preliminaries from persistent homological algebra and topological data analysis. Readers familiar with these topics can safely skip this section.

In Section 2, we introduce the RedZeD algorithm by first providing a theoretical result on replacing a filtration by a quasi-isomorphic sequence, and then extracting an algorithm, called "Naive RedZeD", from it. Naive RedZeD is a naive implementation of reducing a filtered chain complex to a sequence with zero differentials, which we use for ease of proving correctness and extracting the interpretation in terms of the standard matrix reduction algorithm. We conclude Section 2 by discussing simple improvements that speed up our algorithm, including explicitly storing inverses of the dictionaries R_p and deleting keys when the corresponding values are zero. We reserve the name RedZeD to refer to the algorithm with these improvements implemented, distinguishing it from the Naive RedZeD algorithm.

In Section 3, we describe active enumeration and how it can be used to speed up computations of Vietoris–Rips persistent homology. In particular, we show how active simplices can be understood in the matrix-theoretic language. This gives RedZeD_VR, a specialization of RedZeD to the Vietoris–Rips filtration. The RedZeD_VR algorithm is implemented in both Julia and C++, and details on the implementation are described in Section 4. We then compare RedZeD_VR to Ripser in Section 5, reporting on relative differences in performance. We conclude in Section 6 with a summary of the paper and a discussion of future directions.

The code for RedZeD_VR is available on

`https://github.com/nkershaw01/RedZeD`

and can also be downloaded as a Python package called `redzed-tda`:

`https://pypi.org/project/redzed-tda/`.

Related work

As mentioned before, the results and algorithms of Section 2 can be viewed as a new (and more general) interpretation of part of the algorithm presented in [DFW14], which concerns persistent homology of

arbitrary simplicial maps (not necessarily inclusions). [DFW14] works with cohomology and keeps track of the cohomological basis using the *annotation* map, which is analogous to the map r described above. The behavior of this algorithm on *elementary inclusions* corresponds directly to the steps performed in RedZeD. We comment on this relationship in detail in Remark 2.9.

In the language of matrix reduction algorithms, the techniques employed by RedZeD include compression along with exhaustive and retrospective reduction. These have been studied extensively and we refer the reader to [BMG+24] for an excellent overview and comparison.

In terms of performance, we compared RedZeD_VR to Ripser [Bau21], which represents the high water mark in computing persistent homology of Vietoris–Rips filtrations. In the implementation of RedZeD_VR, we also borrow several ideas from Ripser, notably its combinatorial numbering system used to encode simplices.

Acknowledgements

As newcomers to the area, we have greatly benefited from conversations with Håvard Bakke Bjerkevik, Daniel Carranza, Tamal Dey, Barbara Giunti, Mike Lesnick, Dmitriy Morozov, and Luis Scoccola. We are grateful to all of them for patiently answering our questions and providing excellent suggestions and insights.

AI Disclaimer

After completing the first implementation, we used Claude Opus 5 to optimize the code; in particular, to create a new implementation of dictionaries, with backward shift deletion instead of using tombstones. We also used Claude to translate our code from Julia to C++.

1 Preliminaries

In this section, we review the necessary background on (persistent) homological algebra. Throughout, we work over a field F, suppressing it from notation, thus, for example, writing Ch for the category of chain complexes and chain maps. Similarly, we write $H_n(X)$ to mean the n-th homology group of a simplicial complex X with coefficients in F, more commonly denoted $H_n(X; F)$.

In the context of persistence, our main object of study will be filtrations of chain complexes. Although we index our filtrations by natural numbers, we could alternatively choose any finite totally ordered set.

Definition 1.1. A (finite) *filtration* C^* of chain complexes is a sequence of inclusions of chain complexes

$$C^0 \subseteq C^1 \subseteq \cdots \subseteq C^n.$$

To make our filtrations well-behaved, we will impose the condition that our filtrations are *elementwise*, i.e., the "difference" between C^i and C^{i+1} is a single generator. To express it in the categorical language, which will be useful to us a little later, we define the algebraic models of the n-sphere and the n-disk for a non-negative integer n. The *algebraic n-sphere* $S(n)$ and the *algebraic n-disk* $D(n)$ is the chain complexes given by

$$S(n)_p = \begin{cases} F & \text{if } n = p \\ 0 & \text{otherwise} \end{cases} \qquad \text{and} \qquad D(n)_p = \begin{cases} F & \text{if } n = p-1, p \\ 0 & \text{otherwise.} \end{cases}$$

For every n, there is a natural inclusion $S(n-1) \hookrightarrow D(n)$. With that as a preamble, we can now define elementwise filtrations.

Definition 1.2. A filtration C^* is *elementwise* if $C^0 = 0$, and for all $1 \leq i \leq n$ C^i can be constructed via a pushout square:

$$\begin{array}{ccc} S(n-1) & \longrightarrow & C^{i-1} \\ \downarrow & & \downarrow \\ D(n) & \longrightarrow & C^{i} \end{array}$$

Explicitly, in an elementwise filtration, we have for every $i \geq 1$ there is a p such that:

$$(C^i/C^{i-1})_n = \begin{cases} F & \text{if } n = p \\ 0 & \text{otherwise.} \end{cases}$$

If we have a choice of basis preserved by the inclusions, we say that the generator x of F in dimension p is the *element added at time* i, and p is called the *dimension* of x. Let C^* be an elementwise filtration, and x a basis element of C^i_p for some i and p. The *filtration time* of x, denoted $\mathrm{filt}(x)$, is the minimal index j such that $x \in C^j_p$.

As chain complexes are typically studied using homology, we use persistence modules to study filtrations of chain complexes. Since our filtrations are indexed by finite totally ordered sets, we will likewise consider persistence modules over finite totally ordered sets, obtained by taking the homology of chain complexes in the filtration.

Definition 1.3. A *persistence module* over a totally ordered set T is a functor $V \colon T \to \mathsf{Vect}_F$ from T, viewed as a category, to the category of F-vector spaces.

Explicitly, a persistence module V over T consists of a collection of F-vector spaces $\{V_i\}_{i \in T}$ together with maps $\varphi^j_i \colon V_i \to V_j$ for all $i \leq j$, such that each $\varphi^i_i = \mathrm{id}$ and for all $i \leq j \leq k$, we have $\varphi^k_i = \varphi^k_j \circ \varphi^j_i$.

An important example of a persistence module is the notion of an interval module:

Example 1.4. Let T be a totally ordered set, and $b \leq d \in T$. The *interval module*, denoted $I[b, d)$, is the persistence module where $I[b, d)_i$ is F for $b \leq i < d$, and 0 elsewhere. For $i \leq j$, $\varphi^j_i = \mathrm{id}$ for $i \in [b, d)$, and the zero map otherwise.

Given a persistence module over a finite set V, the following classical theorem of P. Gabriel allows us to decompose it into interval modules in a canonical way:

Theorem 1.5 (Gabriel, [Gab72])**.** *For a finite totally ordered set T and a persistence module V over T, there exists a unique decomposition of V into a direct sum of interval modules:*

$$V \cong \bigoplus_J I[b_j, d_j). \qquad \square$$

Definition 1.6. Let C^* be a filtration of chain complexes, and n a natural number. The *persistence diagram* of C^* in dimension n is the multiset of pairs $\mathsf{dgm}_n(C^*) = \{[b_j, d_j)\}_{j \in J}$ associated to the unique decomposition $H_n(C^*) \cong \bigoplus_J I[b_j, d_j)$. A *persistence pair* is an element of $\mathsf{dgm}_n(C^*)$.

Note that $\mathsf{dgm}_n(C^*)$ is a multiset, rather than just a set, because an interval $I[b, d)$ may appear multiple times in the decomposition. The computation of the persistence diagram of a filtration of chain complexes is often referred to as *persistent homology*. The standard method to compute the persistence diagram is the *persistence pairing algorithm*, [ZC05, dSMVJ11, Bau21], which is a matrix reduction algorithm.

The input to the persistence pairing algorithm is the *filtration boundary matrix* M. The matrix M is an $n \times n$ matrix, with rows and columns representing elements in the filtration, ordered by the order in which they appear. Let x and y denote elements in the filtration. We write M_y to mean the column corresponding to the element y, and M_{xy} to be the entry in the row corresponding to x and the column corresponding to y. If $x \in \partial y$, the entry M_{xy} corresponds to the coefficient of x in ∂y, and otherwise $M_{xy} = 0$. This way, the column M_y represents ∂y.

The matrix M is reduced column by column, from left to right. A column M_y is reduced only by adding scalar multiples of columns to the left of it, and it is reduced until it is either zero or has a pivot unique from all previous columns. Once the entire matrix is reduced, the persistence pairs can be read off from the reduced matrix. If an entry M_{xy} is a pivot of column y, this corresponds to a persistence pair (x, y). A zero column M_y such that y does not appear as the pivot row of any other column corresponds to a pair (y, ∞).

Let C^* be an elementwise filtration and suppose that C^i was obtained from C^{i-1} by adding a generator x of dimension p. This implies that either: the dimension of the image of ∂_p increased by 1, or the dimension of the kernel of ∂_p increased by 1. In the former case, $\dim H_{p-1}(C^i) = \dim H_{p-1}(C^{i-1}) - 1$ and we call x a *death element*. In the latter case, $\dim H_{p-1}(C^i) = \dim H_{p-1}(C^{i-1}) + 1$ and we call x a *birth element*. Equivalently, x is a birth element if the column corresponding to x in the reduced matrix is zero, and a death element otherwise.

Persistent homology is most commonly used to compute Vietoris–Rips homology of finite metric spaces. The Rips complex construction associates a filtration of simplicial complexes to a finite metric space. However, the construction is more general than that; namely, it works for an arbitrary set X equipped with a symmetric distance function d, i.e., a function $d\colon X \times X \to [0, \infty)$ such that for all $x, y \in X$, we have $d(x, x) = 0$ and $d(x, y) = d(y, x)$. Compared to the definition of a metric, we are therefore dropping the triangle inequality. Such generality is useful in a variety of applications, including those considered in [KK25].

Definition 1.7. Let (X, d) be a finite set equipped with a distance function, and $r \in [0, \infty)$. The *Vietoris–Rips complex* at scale r is the simplicial complex $\mathsf{VR}_r(X)$ with:

- $\mathsf{VR}_r(X)_0 = X$
- $\mathsf{VR}_r(X)_n = \{\{x_0, \ldots, x_n\} \mid d(x_i, x_j) \leq r \text{ for all } 0 \leq i, j \leq n\}$

Letting r vary, we can form a filtration of simplicial complexes called the *Vietoris–Rips filtration*, denoted $\mathsf{VR}(X)$.

The complex $\mathsf{VR}_r(X)$ can also be viewed as the clique complex on the graph $G_r(X)$, which has as vertices X and an edge $x \sim y$ if $d(x, y) \leq r$. With this viewpoint, we can also generalize the Vietoris–Rips filtration to any filtration of graphs.

Let C be the functor sending a simplicial complex K to the chain complex $C(K)$, which in dimension n is the F-vector space generated by the set of n-simplices. Applying this to each element of the filtration $\mathsf{VR}(X)$, we get a filtration of chain complexes $C(\mathsf{VR}(X))$.

Definition 1.8. The *Vietoris–Rips persistence diagram* of a set (X, d) equipped with a symmetric distance function is the persistence diagram $\mathsf{dgm}_{\mathsf{VR}}(X) = \mathsf{dgm}(C(\mathsf{VR}(X))$.

The Vietoris–Rips filtration is not necessarily an elementwise one, but it can be refined to one. To do this, we impose (arbitrarily) a total order on the elements of X. Then, we can order the simplices on the complete simplicial complex on X lexicographically according to

1. the diameter (i.e., maximum length of an edge in the simplex) with $\sigma < \tau$ if $\operatorname{diam}(\sigma) < \operatorname{diam}(\tau)$;
2. the dimension with $\sigma < \tau$ if $\operatorname{diam}(\sigma) = \operatorname{diam}(\tau)$, and $\dim(\sigma) < \dim(\tau)$;
3. ordering on the vertices with $\sigma \leq \tau$ if their diameters and dimensions are equal, and the list of vertices of σ are lexicographically less than or equal to the list of vertices of τ.

We can then form a filtration indexed by the natural numbers, where at time i we add the i-th simplex according to the order. We call this filtration the *lexicographically refined Vietoris–Rips filtration* and denote it $\mathsf{VR}_{\mathsf{ref}}(X)$, thus making the filtration $C(\mathsf{VR}_{\mathsf{ref}}(X))$ elementwise as well. In practice, one thus

computes $\mathsf{dgm}(C(\mathsf{VR}_{\mathsf{ref}}(X))$ and uses it to reconstruct $\mathsf{dgm}_{\mathsf{VR}}(X)$. This is done by replacing each (i, j) with $(\mathrm{diam}(\sigma_i), \mathrm{diam}(\sigma_j))$, where σ_i and σ_j are the simplices added at times i and j, and then removing all intervals of length 0. Because of this, we will always assume that all the Vietoris–Rips filtrations are already refined, and will omit the subscript $(-)_{\mathsf{ref}}$ going forward.

Similar to the terminology for chain complexes, in the refined filtration, we call a simplex σ a *birth simplex* if its corresponding element in the filtration is a birth element. We call a σ a *death simplex* if its corresponding element is a death element.

2 The RedZeD algorithm

In this section, we introduce our general purpose algorithm RedZeD for computing persistent homology of an arbitrary elementwise filtration of chain complexes over $\mathbb{F}_2$. The name RedZeD is a syllabic abbreviation and stands for **Red**uction to **Ze**ro **D**ifferentials. Although stated for $\mathbb{F}_2$, the algorithm can be easily modified to work over an arbitrary field, which we comment on in Remark 2.8. Note that while we are referring to the algorithm presented in this section as RedZeD, in Section 3 we introduce a further improvement, called *active enumeration*, which specializes RedZeD to Vietoris–Rips filtration and yields a new algorithm RedZeD_VR. On its own, the RedZeD algorithm does not offer an improvement over existing methods, and is in general significantly slower. Without RedZeD, however, the active enumeration speedup is not possible, making RedZeD a necessary component.

We do not claim any originality in developing the RedZeD algorithm; as indicated above, it is merely a rephrasing of part of the algorithm in [DFW14], which we comment on in more detail in Remark 2.9. Similarly, while RedZeD might not at first appear to be performing matrix reductions of the persistence pairing algorithm, it can be translated into the matrix-theoretic language. Upon such translation, discussed later in the section, it recovers several of the well-known improvements to the standard algorithm, namely, compression [BKR14] along with exhaustive [EZ03, ZC05, EO20] and retrospective reduction [BMG+24].

In the first part of this section, we introduce Naive RedZeD, the version of the algorithm that directly follows from Theorem 2.1, and prove its correctness. In the second part of the section, we translate Naive RedZeD to the language of matrices and discuss the comparisons with existing improvements to the standard algorithm. In the final part of this section, we discuss two minor improvements to Naive RedZeD. When these improvements are implemented, we call the algorithm RedZeD.

Recall that a chain map $f\colon C \to D$ between chain complexes is a *quasi-isomorphism* if it induces isomorphisms $H_n(C) \to H_n(D)$ for every non-negative integer n. The key theorem underlying our algorithm says that an elementwise filtration of chain complexes can be replaced by quasi-isomorphic one in which all chain complexes have zero differentials. More precisely, it is the procedure given in the proof of this statement that underpins our algorithm.

Theorem 2.1. *Let F be a field and $C^*\colon 0 = C^0 \hookrightarrow C^1 \hookrightarrow \cdots \hookrightarrow C^m$ be an elementwise filtration of chain complexes over F. Then there exists a diagram*

$$\begin{array}{ccccccc} C^0 & \longrightarrow & C^1 & \longrightarrow & \cdots & \longrightarrow & C^m \\ {\scriptstyle r^0}\downarrow{\scriptstyle\simeq} & & {\scriptstyle r^1}\downarrow{\scriptstyle\simeq} & & & & {\scriptstyle r^n}\downarrow{\scriptstyle\simeq} \\ A^0 & \longrightarrow & A^1 & \longrightarrow & \cdots & \longrightarrow & A^n \end{array}$$

where each r^i is a quasi-isomorphism, and the maps $\partial^i_j : A^i_j \to A^i_{j-1}$ are the zero map for all i and j.

The proof of the theorem depends on a stability property of quasi-isomorphisms in the category of chain complexes. Recall that the category of chain complexes carries the (injective) model structure whose cofibrations are the monomorphisms and weak equivalences are the quasi-isomorphisms [Hov99, Thm. 2.3.13]. Since all objects are injectively cofibrant, this model structure is left proper [Hir03, Prop. 13.1.2], i.e., pushouts of quasi-isomorphisms along monomorphisms are again quasi-isomorphisms.

Lemma 2.2. *Let $f\colon C \to D$ be a quasi-isomorphism and $C \hookrightarrow C'$ a levelwise injective morphism. Let f' be the pushout of f along this inclusion:*

$$\begin{array}{ccc} C & \hookrightarrow & C' \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle f'} \\ D & \longrightarrow & D' \end{array}$$

Then f' is again a quasi-isomorphism. □

With the lemma in hand, we can now give the proof of Theorem 2.1.

Proof of Theorem 2.1. Since each C^i is quasi-isomorphic to A^i, we must have that $A^i \cong H_*(C^i)$, viewed as a chain complex with zero differential. The important step is constructing the maps r^i.

We proceed by induction. We have that $C_0 = A_0 = 0$, and $r^0 = \mathrm{id}_0$. Now suppose we have $r^{i-1}\colon C^{i-1} \to A^{i-1}$. Since C^* is elementwise, C^i is obtained via a pushout

$$\begin{array}{ccc} S(p-1) & \longrightarrow & C^{i-1} \\ \downarrow & & \downarrow \\ D(p) & \longrightarrow & C^i \end{array}$$

Let x be the image of a generator under the map $D(p)_p \to C^i_p$. We do case analysis on the value of $r^{i-1}_{p-1}\partial x$.

Case 1: $r^{i-1}_{p-1}\partial x = 0$. In this case, define $r^i\colon C^i \to A^i$ by the pushout:

$$\begin{array}{ccc} C^{i-1} & \xrightarrow{r^{i-1}} & A^{i-1} \\ \downarrow & & \downarrow \\ C^i & \xrightarrow{r^i} & A^i \end{array}$$

By Lemma 2.2 we see that since r^{i-1} is a quasi-isomorphism, $C^{i-1} \to C^i$ is levelwise injective, r^i must also be a quasi-isomorphism. We need to verify that A^i has zero differentials. The only thing to check is that $\partial(r^i_p x) = 0$, but this follows by unwinding the definition:

$$\partial(r^i_p x) = r^i_{p-1}\partial x = r^{i-1}_{p-1}\partial x = 0$$

with the last equality given by assumption. In this case, what is happening is that adding in x did not change the rank of ∂_p, i.e., $\mathrm{rank}(\partial^{i-1}_p) = \mathrm{rank}(\partial^i_p)$. Thus we must have that $\dim(\ker(\partial^i_p)) = \dim(\ker(\partial^{i-1}_p)) + 1$. We are therefore increasing the dimension of the p-th homology group by 1, so to form A^i from A^{i-1} we must add an element in dimension p. The map r^i is then identical to r^{i-1} but sends x to this new element in A^i_p.

Case 2: $r^{i-1}_{p-1}\partial x \neq 0$. We define A^i

$$\begin{array}{ccccc} S(p-1) & \longrightarrow & C^{i-1} & \xrightarrow{r^{i-1}} & A^{i-1} \\ \downarrow & & \downarrow & & \\ D(p) & \longrightarrow & C^i & \overset{r^i}{\searrow} & \downarrow \\ \downarrow & & & & \\ 0 & & \longrightarrow & & A^i \end{array}$$

with r^i given by the induced map. To see that r^i is a quasi-isomorphism, we take the pushout in:

$$\begin{array}{ccc} C^{i-1} & \xrightarrow{r^{i-1}} & A^{i-1} \\ \downarrow & & \downarrow \\ C^i & \longrightarrow & P \end{array}$$

thus writing r^i as a composite of $C^i \to P \to A^i$. Since $C^{i-1} \to C^i$ is a monomorphism, Lemma 2.2 tells us that the first factor is a quasi-isomorphism as a pushout of r^{i-1} along a monomorphism. Moreover, the pushout pasting lemma implies that

$$\begin{array}{ccccc} D(p) & \longrightarrow & C^i & \longrightarrow & P \\ \downarrow & & & & \downarrow \\ 0 & & \longrightarrow & & A^i \end{array}$$

is a pushout as well. The top map is a monomorphism by assumption, and $D(p) \to 0$ is a quasi-isomorphism, so by Lemma 2.2, we obtain that $P \to A^i$ is a quasi-isomorphism. □

Remark 2.3. The requirement that we are over a field is necessary for Theorem 2.1. For example, consider the chain complex

$$\cdots \to 0 \to \mathbb{Z}\langle y_1, y_2 \rangle \to \mathbb{Z}\langle x \rangle$$

where $y_1 \mapsto 2x$ and $y_2 \mapsto x$. Suppose we have a filtration on the generators $x \to y_1 \to y_2$.

For the theorem to hold, we in particular need a commutative square:

$$\begin{array}{ccc} C^2 & \longrightarrow & C^3 \\ {\scriptstyle r^2}\downarrow & & \downarrow{\scriptstyle r^3} \\ A^2 & \longrightarrow & A^3 \end{array}$$

where each A^i has zero differentials and each r^i is a quasi-isomorphism. We have that A^2 must be

$$\cdots \to 0 \to 0 \to \mathbb{Z}/2$$

and that A^3 must be

$$\cdots \to 0 \to \mathbb{Z} \to 0$$

Thus, for such a commutative square to exist, since $r^2(y_1) = 0$, we must have that $r^3(y_1) = 0$. But we need r^3 to be a quasi-isomorphism, so it in particular must map the generator of $H_1(C^3)$ to the generator of $H_1(A^3)$. But the generator of $H_1(C^3)$ is $y_1 - 2y_2$, and we get that $r^3(y_1 - 2y_2) = -2r^3(y_2)$. Regardless of what $r^3(y_2)$ is, we know that $2r^3(y_2)$ cannot be a generator of $H_1(A^3)$.

While Theorem 2.1 works over an arbitrary field F, from this point on we set $F = \mathbb{F}_2$. We begin by fixing notation specific to $\mathbb{F}_2$.

Notation 2.4. For vector spaces, we assume we have a choice of basis. A vector x is expressed as a set $\{x_1, \ldots, x_k\}$, which is the set of basis elements whose coefficient is nonzero when expressing x as a linear combination. For basis elements, we occasionally write x_i to mean $\{x_i\}$. Consequently, vector addition $x + y$ is then performed via symmetric difference $\{x_1, \ldots, x_k\} \triangle \{y_1, \ldots, y_\ell\}$, and instead of $\sum x_i$ we write $\bigtriangleup\{x_{i_1}, \ldots x_{i_k}\}$. For ease of future reference, we summarize this translation in the following table:

F	$\mathbb{F}_2$
$x \in F^n$	$\{x_1, \ldots, x_k\}$
0	$\varnothing$
$x + y$	$\{x_1, \ldots x_k\} \triangle \{y_1 \ldots, y_\ell\}$
$\sum_{i \in I} x_i$	$\bigtriangleup_{i \in I} \{x_{i_1}, \ldots, x_{i_k}\}$

We now describe how to turn the proof of Theorem 2.1 into an algorithm for computing persistent homology of the filtration. The key insight is that A^i depends only on A^{i-1}, r^{i-1}, and the element x (along with its boundary ∂x) that gets added at time i. We then compute $r^{i-1}\partial x$, and the different cases tell us how to construct A^i as well as r^i. After each i, we thus only ever need to store r^i and A^i. We can store r_p^i using a dictionary R_p, which has as keys all basis elements x of C_p^i that are births, and as values $R_p[x] = r_p^i(x)$. We can also recover the basis of A_p^i from this dictionary, by finding all keys x such that $R_p[x] = \{x\}$. At every stage in the algorithm, we update R_p to reflect r_p^i.

To compute persistent homology up to dimension n, the algorithm starts by initializing empty dictionaries R_p for every $0 \leq p \leq n$. For each element x that gets added, we need to first compute $r_{p-1}^{i-1}\partial x$. To do this, we initialize a set $r\partial x = \varnothing$ and then for every $y \in \partial x$, we check if $y \in \text{keys}(R_{p-1})$. If not, this means that $r_{p-1}^{i-1} y = 0$, so we skip it. Otherwise, we update $r\partial x$ by adding $R_{p-1}[y]$ over $\mathbb{F}_2$, i.e., taking the symmetric difference $r\partial x \triangle R_{p-1}[y]$.

Once we have computed $r\partial x$, we break into the two cases as in Theorem 2.1. Case 1 is if $r\partial x = \varnothing$. In this case x is a birth element, and gets added as a basis element into A_p^i with $r_p^i x = x$. To signify this, we set $R_p[x] = \{x\}$.

Case 2 is if $r\partial x = \{y_1, \ldots, y_k\}$. In this case x is a death element which kills the linear combination $\{y_1, \ldots, y_k\}$, consisting of at least one element, where we assume that $\text{filt}(y_k) > \text{filt}(y_j)$ for all $1 \leq j \leq k-1$. To get the correct decomposition, we store $(\text{filt}(y_k), \text{filt}(x))$ as a persistence pair of dimension $p-1$. The vector space A_{p-1}^i is then given by quotienting $\{y_1, \ldots, y_k\}$ to zero, and the new basis is given by deleting y_k. Thus for any z, if $r_{p-1}^{i-1} z = B$, where $y_k \in B$, then $r_{p-1}^i z = B \triangle \{y_1, \ldots, y_k\}$. This replaces (mod 2) $y_k \in B$ with $\{y_1, \ldots, y_{k-1}\}$. To update R_{p-1}, we loop though all $z \in \text{keys}(R_{p-1})$ such that $y_k \in R_p[z]$, and set $R_{p-1}[z] = R_{p-1}[z] \triangle \{y_1, \ldots, y_k\}$

As stated, the RedZeD algorithm works for an arbitrary element-wise filtration of chain complexes in $\mathsf{Ch}(\mathbb{F}_2)$. It assumes the filtration comes with a function `filtration_enumerator` which, when called, returns the index of the element in the filtration, the dimension of the element, and a list of the elements in its boundary. Pseudocode for the algorithm is given below. We call this version Naive RedZeD, since a few further improvements can be made, which we describe at the end of this section.

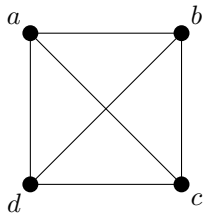
Algorithm 1 Naive RedZeD

Input 1: An elementwise filtration given in the form of `filtration_enumerator`
Input 2: A natural number n
Output: The persistence pairs of the filtration up to dimension n

Initialize empty lists $P_0, \ldots, P_n$
Initialize dictionaries $R_0, \ldots, R_n$
for $i = 1$ to length of filtration **do**
    x, p, ∂x = `filtration_enumerator`(i)
    Initialize $r\partial x = \varnothing$
    for $y \in \partial x$ **do**
        if $y \in \text{keys}(R_{p-1})$ **then**
            $r\partial x = r\partial x \triangle R_{p-1}[y]$
        end if
    end for
    if $r\partial x = \varnothing$ **then**
        $R_p[x] = \{x\}$ if $p \leq n$
    else
        Let $r\partial x = \{y_1, \ldots, y_k\}$ with $\text{filt}(y_k) > \text{filt}(y_j)$ for all $1 \leq j \leq k-1$
        for $z \in \text{keys}(R_{p-1})$ such that $y_k \in R_{p-1}[z]$ **do**
            $R_{p-1}[z] = R_{p-1}[z] \triangle r\partial x$
        end for
        Append $(\text{filt}(y_k), i)$ to P_{p-1}
    end if
end for
for $p = 0$ to n **do**
    for $x \in \text{keys}(R_p)$ such that $R_p[x] = \{x\}$ **do**
        Append $(\text{filt}(x), \infty)$ to P_p
    end for
end for
return $P_0, \ldots, P_n$

Before discussing the correctness of the algorithm, we first consider the following example:

Example 2.5. Suppose that an elementwise filtration chain complexes arises from an elementwise filtratation of simplicial complexes of dimension at most 2 with four points, a, b, c, and d.



The refined linear ordering of the filtration is given by:

$$a \to b \to c \to d \to ab \to bc \to cd \to ad \to ac \to acd \to abc \to bd \to bcd \to abd.$$

In other words, it is a filtration of the boundary of the 3-simplex.

We are interested in computing homology in dimensions 0 and 1. The following table represents the operations performed in the execution of Naive RedZeD at each step of the filtration, and a detailed description is given below.

Simplex x	∂x	$r\partial x$	R_0	R_1	Pairs
a	0	0	$a \to a$		
b	0	0	$a \to a$ $b \to b$		
c	0	0	$a \to a$ $b \to b$ $c \to c$		
d	0	0	$a \to a$ $b \to b$ $c \to c$ $d \to d$		
ab	$a+b$	$a+b$	$a \to a$ $b \to a$ $c \to c$ $d \to d$		(b, ab)
bc	$b+c$	$a+c$	$a \to a$ $b \to a$ $c \to a$ $d \to d$		(b, ab), (c, ac)
cd	$c+d$	$a+d$	$a \to a$ $b \to a$ $c \to a$ $d \to a$		(b, ab), (c, ac), (d, cd)
ad	$a+d$	$a+a=0$	$a \to a$ $b \to a$ $c \to a$ $d \to a$	$ad \to ad$	(b, ab), (c, ac), (d, cd)
ac	$a+c$	$a+a=0$	$a \to a$ $b \to a$ $c \to a$ $d \to a$	$ad \to ad$ $ac \to ac$	(b, ab), (c, ac), (d, cd)
acd	$bd+ad+ac$	$ad+ac$	$a \to a$ $b \to a$ $c \to a$ $d \to a$	$ad \to ad$ $ac \to ad$	(b, ab), (c, ac), (d, cd), (ac, acd)
abc	$bc+ac+ab$	ad	$a \to a$ $b \to a$ $c \to a$ $d \to a$		(b, ab), (c, ac), (d, cd), (ac, acd), (ad, abc)
bd	$b+d$	$a+a=0$	$a \to a$ $b \to a$ $c \to a$ $d \to a$	$bd \to bd$	(b, ab), (c, ac), (d, cd), (ac, acd), (ad, abc)
bcd	$cd+bd+bc$	bd	$a \to a$ $b \to a$ $c \to a$ $d \to a$		(b, ab), (c, ac), (d, cd), (ac, acd), (ad, abc), (bd, bcd)
abd	$bd+ad+ab$	0	$a \to a$ $b \to a$ $c \to a$ $d \to a$		(b, ab), (c, ac), (d, cd), (ac, acd), (ad, abc), (bd, bcd)

The last step in the algorithm is to add an interval (a, ∞) since it is the only remaining value in R_0.

Each of the first four steps adds a single vertex, which becomes a generator in H_0, and thus $R_0[x] = \{x\}$ in each case. When ab is added, we have $\partial(ab) = r\partial(ab) = a+b$. Since b is younger, we set $R_0[b] = \{a\}$ and add the pair (b, ab). When bc is added, we have $\partial(bc) = b + c$, but since $R_0[b] = \{a\}$, we get $r\partial bc = a + c$. Since c is younger, we set $R_0[c] = a$ and add (b, bc) to the set of pairs. A similar process is repeated for cd.

When ad is added, we have $\partial(ad) = a + d$, and since $R_0[d] = \{a\}$, $r\partial(ad) = a + a = 0$. Thus ad is a birth, and we set $R_1[ad] = \{ad\}$. When ac is added, it is also a birth, and we set $R_1[ac] = \{ac\}$. When acd is added, we have that $\partial(acd) = bd + ad + ac$. We have that $R_1[ac] = \{ac\}$, $R_1[ad] = \{ad\}$, and $bd \notin \text{keys}(R_1)$, so $r\partial abd = ad + ac$. We thus set $R_1[ac] = \{ac\} \triangle \{ad, ac\} = \{ad\}$, and add (ac, acd) as a pair. When abc is added we get that $\partial(abc) = bc + ac + ab$, and thus $r\partial(abc) = R_1[ac] = ad$. We thus update $R_1[ac] = \{ad\} \triangle \{ad\} = \varnothing$, so we delete $R_1[ac]$. Identically we delete $R_1[ad]$, and add (ad, abc) as a pair. The remaining steps in the algorithm are all similar cases to the steps above.

We are now ready to state and prove correctness of Naive RedZeD.

Theorem 2.6. *Naive* RedZeD *correctly computes all persistence pairs for an elementwise filtration C^*.*

The proof requires the following lemma:

Lemma 2.7. *For an elementwise filtration, after every step i in the filtration and for every $0 \leq p \leq n$, R_p correctly encodes r_p^i, in that for every basis element $x \in C_p^i$:*

$$r_p^i(x) = \begin{cases} R_p[x] & \text{if } x \in \text{keys}(R_p) \\ 0 & \text{if } x \notin \text{keys}(R_p) \end{cases}$$

Proof. We proceed by induction. The base case is immediate, since $C_p^0 = 0 = A_p^0$ and R_p and R_p^{-1} are empty. Now suppose that for all p, the dictionary R_p correctly encodes r_p^i. Suppose at time i an element of dimension p is added. We have two cases:

Case 1: $r_{p-1}^{i-1}\partial x = 0$. In this case, we know that we land in case 1 of the algorithm since $r\partial x$ computed by the algorithm is equal to the symmetric difference of $R_{p-1}[y]$ over all $y \in \partial x$. This in turn is equal to the addition mod 2 of $r_{p-1}^{i-1}y$ over all $y \in \partial x$ since by assumption R_{p-1} correctly encodes r_{p-1}^{i-1}. The element $y \in \partial x$ that are not keys in R_{p-1} are such that $r_{p-1}^{i-1}y = 0$.

By Theorem 2.1, the vector space A^i is obtained by adding x as a new basis element to A_p^{i-1} and setting $r_p^i(x) = x$. This is exactly what the algorithm does, by setting $R_p[x] = \{x\}$. Since r_p^i remains the same on all other basis elements, and we did not change R_p aside from adding x, we are done.

Case 2: $r_{p-1}^{i-1}x \neq 0$. Again by assumption since R_{p-1} encodes r_{p-1}^{i-1}, we get that $r\partial x$ computed by the algorithm is equal to $r_{p-1}^{i-1}x$, so we land in case 2 of the algorithm. Suppose $r\partial x = \{y_1, \ldots, y_k\}$ for basis elements $y_1, \ldots, y_k$, where y_k is the youngest element in the set. In this case, only A_{p-1}^{i-1} is changed, with $r\partial x = \{y_1, \ldots, y_k\}$ quotiented to zero, and y_k removed from the basis. Suppose z is a basis element of C_{p-1}^{i-1} such that $r_{p-1}^{i-1} = B$, where $y_k \in B$. Then $r_{p-1}^i(z) = B \triangle \{y_1, \ldots, y_k\}$. Thus for every z such that $y_k \in R_{p-1}[z]$, we need to replace y_k with $y_1, \ldots, y_{k-1}$, which is exactly done by lines 15-17 in the algorithm. Thus after all $z \in \text{keys}(R_{p-1})$ with $y_k \in R_{p-1}[z]$ are processed, R_{p-1} correctly encodes r_{p-1}^i. Since only dimension $p-1$ was changed, this still holds for all other dimensions. □

We now prove Theorem 2.6.

Proof of Theorem 2.6. Since each r_i is a quasi-isomorphism and each A^i has zero differentials, in each dimension p the sequence

$$A_p^0 \to A_p^1 \to \cdots \to A_p^n$$

is the persistence module associated with the p-th homology of C^*. We thus need to show that the decomposition given by RedZeD is a decomposition of this persistence module.

For $i < j$, we want to understand the transition maps $A^i \to A^j$, which we will denote s_i^j. It suffices to determine it degreewise on basis elements. Suppose $x \in A_p^i$ is a basis element, i.e., a key in R_p such that after time i, $R_p[x] = \{x\}$. Then we can view x as an element of C_p^i such that $r_p^i(x) = x$. By commutativity of the diagram

$$\begin{array}{ccc} C^i & \hookrightarrow & C^j \\ {\scriptstyle r^i}\downarrow & & \downarrow{\scriptstyle r^j} \\ A^i & \xrightarrow{s_i^j} & A^j \end{array}$$

we get that $s_j^i(x) = r_p^j(x)$. After step j of the algorithm, by Lemma 2.7, we thus have that:

$$s_j^i(x) = \begin{cases} R_p[x] & x \in \text{keys}(R_p) \\ 0 & x \notin \text{keys}(R_p). \end{cases}$$

Let B^* be the persistence module given by the algorithm. That is, $B^* = \bigoplus_I [b_i, d_i)$, where $\{(b_i, d_i)\}_I$ are the pairs given by the algorithm. We want to show that $B^* \cong A^*$. Note that since B^* may contain infinite intervals, we are viewing A^* as infinite as well, being the identity at each step after the final filtration time.

We can choose a basis for B^i to be the set of all x, such that x is a birth element corresponding to an interval (j, k) where $j \leq i < k$. Since the basis for A^i is exactly that set of elements, we see that $A^i \cong B^i$ for all i. The natural map exhibiting this isomorphism need not be an isomorphism $A^* \cong B^*$, however, as it may fail the necessary commutativity conditions.

Instead, we need to choose a new basis for A^i. We do this by replacing each basis element x with a new basis element as follows. Suppose x is a basis element. If $s_i^j x = x$ for all $j \geq i$, then we do not replace x, and define the value of the map $A^i \to B^i$ at x to be x. Otherwise, suppose j is minimal such that $s_i^j x = \{y_1, \ldots, y_k\}$ for $x \notin \{y_1, \ldots, y_k\}$. Then replace x in the basis with $\{y_1, \ldots, y_k, x\}$. Note that by construction, x is younger than all of $y_1, \ldots, y_k$, so after we do this we still maintain a basis. For the same reason, this is also well defined, since each $y_1, \ldots, y_k$ being older than x ensures they are present at time i. Then define $A^i \to B^i$ to send $\{y_1, \ldots, y_k, x\}$ to x. Since this map is an isomorphism on basis elements under this new basis, it is still an isomorphism. It suffices to show that for all i the square

$$\begin{array}{ccc} A^i & \longrightarrow & A^{i+1} \\ \downarrow & & \downarrow \\ B^i & \longrightarrow & B^{i+1} \end{array}$$

commutes.

Since C^* is an elementwise filtration, either a new basis element gets added into A^{i+1} or a generator dies. If a new basis element x gets added in at time $i + 1$, then $A^{i+1} \to B^{i+1}$ will be the same map as $A^i \to B^i$ on all the basis elements of A^i, so we are done. Now suppose $s_i^{i+1} x = \{y_1, \ldots, y_k\}$. Then $\{y_1, \ldots, y_k, x\}$ is a basis element of A_i that gets mapped to $x \in B^i$. Moreover, an element z must have been added at time $i + 1$ such that $r\partial z = \{y_1, \ldots, y_k, x\}$. Thus $(\text{filt}(x), i + 1)$ gets stored as a pair by the algorithm, and thus x gets mapped to 0 by $B^i \to B^{i+1}$. We also have that

$$s_i^{i+1}\{y_1, \ldots, y_k, x\} = \{y_1, \ldots, y_k, x\} \triangle \{y_1, \ldots, y_k, x\} = \varnothing$$

So the square commutes for this basis element. Note that this is the only basis element containing x, since x cannot appear in $r\partial z$ for any future z. Thus for all other basis elements, $A^i \to A^{i+1}$ is the identity, and hence the square commutes. □

Remark 2.8. The RedZeD algorithm can be modified to work over an arbitrary field F. To do this, the values of the dictionaries R_p need to be weighted sets, with the weights being scalars in F. For $r\partial x = a_1 y_1 + \cdots + a_k y_k$, instead of taking the symmetric difference $R_p[z] \triangle \{y_1, \ldots, y_k\}$ for all $z \in R_p^{-1}[y_k]$, one must subtract the weighted set $\{a_1 y_1, \ldots, a_k y_k\}$ from $R_p[z]$ with a sufficient coefficient to cancel the y_k term in $R_p[z]$.

Finally, we comment on the relation between Naive RedZeD and the algorithm found in [DFW14].

Remark 2.9. For elementwise filtrations of chain complexes arising from filtrations of simplicial complexes, our maps $r_p^i \colon C_p^i \to A_p^i$ recover the notion of an *annotation* $a \colon K(p) \to \mathbb{F}_2^g$ used in [DFW14]. Specifically, if K^i is the i-th simplicial complex in a filtration and $C(K^i)$ is the chain complex generated by simplices, the map $r_p^i \colon C_p(K^i) \to A_i$ can be viewed as an annotation $K(p) \to A_i \cong \mathbb{F}_2^g$, where $K^i(p)$ is the set of p-simplices in K^i. In particular, this annotation is valid in the sense of [DFW14, Def. 3.2], and is the annotation used in [DFW14, §4.1] for elementary inclusions. For a simplex σ added at time i, our cases of $r\partial\sigma = 0$ and $r\partial\sigma \neq 0$ are the same as cases $a\partial\sigma = 0$ and $a\partial\sigma \neq 0$ of [DFW14]. While the exact implementation differs, Naive RedZeD's updates to R_p are analogous to the updates performed in [DFW14] to the annotation. In this sense, RedZeD can be viewed as a different perspective on and a generalization of a part of the [DFW14] algorithm.

Matrix interpretation

The formulation of Naive RedZeD given above conceals its relation to the persistence pairing algorithm. To elucidate this relation, we will translate Naive RedZeD into the matrix-theoretic language and explain the specific speedups uncovered by this translation.

To begin, let us briefly review the standard persistence pairing algorithm, described in greater detail in Section 1. It works with the filtration boundary matrix M, and applies a reduction algorithm, reducing columns left to right. To reduce a given column with index M_x, columns M_y with $\mathrm{filt}(y) < \mathrm{filt}(x)$ are added to M_x, until it either has a pivot distinct from all previous columns or is zero. After this reduction is done for all columns, the persistence pairs can be read off from the pivot entries. A pivot entry (x, y) in corresponds to a persistence pair $(\mathrm{filt}(x), \mathrm{filt}(y))$ of dimension $\dim(x)$, and tells us that the M_x column of gets reduced to zero. The infinite intervals may be read off by finding all zero columns M_x that do not appear in any of the pairs (x, y). If x is such an element, then $(\mathrm{filt}(x), \infty)$ is an infinite interval in the diagram.

Of relevance to us are three speedups that can be applied to the algorithm, namely, compression, exhaustive reduction, and retrospective reduction. These were studied in detail in [BMG$^+$24] and we refer interested readers to that reference for a comprehensive overview.

1. *Compression* [BKR14] is the process of deleting rows corresponding to death elements once they are identified. These rows cannot affect the reduction and can therefore be omitted.

2. *Exhaustive reduction* [EO20, EZ03, ZC05] reduces each column even after finding a unique pivot by adding previous columns that have pivot entries in positions where the current column has nonzero entries. Its purpose is to sprasify the matrix during reduction.

3. *Retrospective reduction* [BMG$^+$24] uses the current column to reduce all previous columns via right-to-left addition and thus zeroes the entire row to the left of the pivot. (This is a slightly different formulation than that in [BMG$^+$24], but is the one relevant to our algorithm.) Retrospective reduction occasionally refers to the combination of retrospective and exhaustive reduction; however, we refer to the two processes separately. Like exhaustive reduction, retrospective reduction has also been used to sparsify the matrix during reduction.

To translate between the language of RedZeD and the persistence pairing algorithm, we first fix some notation.

Notation 2.10. We write $y \in M_x$ to mean that M_x has a nonzero entry in the row corresponding to y. If y is a birth element that has been killed, this corresponds to a unique column, denoted M^y, that has pivot y (again, meaning the pivot entry is in the row corresponding to y).

The main connection between RedZeD and the standard algorithm comes from the following proposition:

Proposition 2.11. *Let C^* be an elementwise filtration, and M the corresponding filtration boundary matrix. Suppose an element x is added at time i. Let M_x be the reduced column in M after step i using compression, and exhaustive and retrospective reductions. Then $r\partial x$ computed by Naive* RedZeD *is exactly the set of nonzero entries in M_x.*

Proof. We prove this using induction; however, we require a stronger inductive hypothesis. Namely, we prove that at a step i:

1. $r\partial x$ consists of the nonzero entries in M_x;
2. for an element y, we have $R_p[y] = \{y\}$ if and only if y is a birth element that does not appear as a pivot of any earlier column M_z;
3. for an element y, we have $y \notin R_p[y] = \{z_1, \dots, z_k\}$ if and only if y appears as a pivot of a column M^y. In this case $\{z_1, \dots, z_k\}$ is the set of nonzero entries that lie above y in M^y.

The base case is empty. The proof in the inductive step proceeds as follows:

- prove 1. assuming that 2. and 3. are true prior to the i-th step;
- prove 2. and 3. hold after the i-th step.

If x is a birth element, by correctness of Naive RedZeD, we get that $r\partial x = 0$, and by correctness of the pairing algorithm M_x gets reduced to zero. In this case, retrospective reduction is not applied, and in Naive RedZeD, we set $R_p[x] = \{x\}$ and change nothing else, so we are done.

Now suppose x is not a birth element. We want to show that $r\partial x$ in Naive RedZeD is the set of all nonzero elements in reduced column M_x after exhaustive reduction and compression.

The first step in reduction is compression, so we first zero all death entries in M_x. In Naive RedZeD, this is done implicitly since death entries are not added into R_p, so they are ignored in the computation of $r\partial x$. We may assume death elements are processed first, so after this step $r\partial x$ is still empty, and the elements in ∂x that have yet to be processed are the nonzero entries that remain in column M_x after compression. We thus may assume that both ∂x and M_x contain no death entries.

Since all columns to the left of M_x have been exhaustively and retrospectively reduced, they contain no entries that are pivots of any other columns. To reduce exhaustively reduce M_x, we thus find all elements $y \in M_x$ that are pivots of previous columns, and cancel them by taking $M_x = M_x \triangle M^y$.

Define

$$I_x = \{y \in \operatorname{keys} R_p \mid y \notin R_p[y]\} \cap \partial x \qquad \text{and} \qquad J_x = \{y \in \operatorname{keys}(R_p) \mid R_p[y] = \{y\}\} \cap \partial x.$$

By the inductive hypothesis, I_x is the set of $y \in \partial x$ that appear as pivots of previous columns, and J_x is the set of $y \in \partial x$ that are births that do not appear as previous pivots. Since we are assuming ∂x contains no deaths, $\partial x = I_x \cup J_x$. Writing M'_x for the column M_x after reduction, we have that (note that the M_x on the left is the reduced column and M_x on the right is unreduced):

$$
\begin{aligned}
M'_x &= (\triangle_{y\in I_x} M^y)\triangle M_x \\
&= (\triangle_{y\in I_x} R_p[y]\triangle\{y\})\triangle\partial x && \text{since } M^y = R_p[y]\triangle\{y\} \text{ by I.H.} \\
&= (\triangle_{y\in I_x} R_p[y])\triangle I_x\triangle\partial x \\
&= (\triangle_{y\in I_x} R_p[y])J_x && \text{since } J_x = \partial x \setminus I_x \\
&= (\triangle_{y\in \partial x} R_p[y]) && \text{since } R_p[y] = \{y\} \text{ for } y \in J_x \text{ and } \partial x = I_x \cup J_x \\
&= r\partial x
\end{aligned}
$$

Thus $r\partial x$ is the set of nonzero entries in M_x after exhaustive and retrospective reduction.

It remains to show that after completion of the i-th step, 2. and 3. still hold. Since x is a death, we have that $r\partial x = \{y_1, \ldots, y_k\}$, so y_k is the pivot of M_x. For every z with $\text{filt}(z) \leq \text{filt}(x)$, if $y_k \in M_z$, we cancel y_k by taking $M_z = M_z\triangle M_x = M_z\triangle r\partial x$. if w is the pivot of M_z, then we have $y_k \in R_p[w]$ by the inductive hypothesis. In Naive RedZeD, we set $R_p[w] = R_p[w]\triangle r\partial x$ in line 16. This is exactly the set of nonzero entries above w in $M^w = M_z$ after taking $M_z = M_z\triangle r\partial x = (R_p[w]\triangle\{w\})\triangle r\partial x$. Thus, condition 3. is still satisfied. For condition 2., note that y_k cannot appear in $R_p[z]$ for any birth z that is still alive (not a pivot of any column), so we are done. □

With this perspective, Naive RedZeD can be viewed as a different implementation of compression with exhaustive and retrospective reduction. Our rephrasing allows for a natural further improvement, which we explore in Section 3. We first describe two minor improvements to this version of RedZeD, which are both more computational than mathematical.

Further improvements

The first change to Naive RedZeD is very minor, meant to save memory and make the forthcoming improvements of Section 3 more natural. The idea is that once a birth element x is such that $R_p[x] = \varnothing$, then $R_p[x]$ will stay empty for the remainder of the algorithm. We can therefore delete the key from the dictionary R_p, and it will not change any of the computations in the algorithm.

The second change comes from the observation that in case 2 of the algorithm, we spend a lot of time looping through $z \in \text{keys}(R_p)$ to check if $y_k \in R_p[z]$, namely line 15 of Algorithm 1. To avoid this loop, we maintain a reverse dictionary R_p^{-1}, which has as keys basis elements of A_p^i, and values $R_p^{-1}[x] = \{y \in \text{keys}(R_p) \mid x \in R_p[y]\}$. Then, instead of looping through $z \in \text{keys}\, R_p$ such that $y_k \in R_p[z]$, we loop through $z \in R_p^{-1}[y_k]$. To maintain R_p^{-1}, it starts as empty, and we update it every time a new element x is added. In case 1, where $r\partial x = 0$, since we set $R_p[x] = \{x\}$, we need to also set $R_p^{-1}[x] = \{x\}$. In case 2, we have that $r\partial x = \{y_1, \ldots, y_k\}$. In this case, we normally loop though all $z \in \text{keys}(R_{p-1})$ and if $y_k \in R_{p-1}[z]$ take $R_{p-1}[z] = R_{p-1}[z]\triangle\{y_1, \ldots, y_k\}$, but instead we loop though $z \in R_{p-1}^{-1}[y_k]$ and for each z take $R_{p-1}[z] = R_{p-1}[z]\triangle\{y_1, \ldots, y_k\}$. We then need to update $R_p^{-1}[y_i]$ for each i. For each $z \in R_p[y_k]$, if beforehand we had that $y_i \in R_p[z]$, now $y_i \notin R_p[z]$, and if beforehand $y_i \notin R_p[z]$, now $y_i \in R_p[z]$. Thus to update $R_p^{-1}[y_i]$, we can take $R_p^{-1}[y_i] = R_p^{-1}[y_i]\triangle R_p^{-1}[y_k]$ for each i. We also then delete $R_p^{-1}[y_k]$ from the dictionary. Full psudeocode with these changes is given below:

Algorithm 2 RedZeD

Input 1: An elementwise filtration given in the form of `filtration_enumerator`
Input 2: A natural number n
Output: The persistence pairs of the filtration

Initialize empty lists $P_0, \dots, P_n$
Initialize dictionaries $R_0, \dots, R_n$, and $R_0^{-1}, \dots, R_n^{-1}$
for $i = 1$ to length of filtration **do**
    x, p, ∂x = `filtration_enumerator`(i)
    Initialize empty set $r\partial x$
    for $y \in \partial x$ **do**
        if $y \in \text{keys}(R_{p-1})$ **then**
            $r\partial x = r\partial x \triangle R_{p-1}[y]$
        end if
    end for
    if $r\partial x = \varnothing$ **then**
        $R_p[x] = \{x\}$ if $p \leq n$
    else
        Let $r\partial x = \{y_1, \dots, y_k\}$ with $\text{filt}(y_k) > \text{filt}(y_j)$ for all $1 \leq j \leq k-1$
        for $z \in R_{p-1}^{-1}[y_k]$ **do**
            $R_{p-1}[z] = R_{p-1}[z] \triangle r\partial x$
            if $R_{p-1}[z] = \varnothing$ **then**
                Delete $R_{p-1}[z]$ from the dictionary
            end if
        end for
        for $j = 1$ to $k-1$ **do**
            $R_{p-1}^{-1}[y_j] = R_{p-1}^{-1}[y_j] \triangle R_{p-1}^{-1}[y_k]$
        end for
        Delete $R_{p-1}^{-1}[y_k]$ from the dictionary
        Append $(\text{filt}(y_k), i)$ to P_{p-1}
    end if
end for
for $m = 0$ to n **do**
    for $x \in \text{keys}(R_p^{-1})$ **do**
        Append $(\text{filt}(x), \infty)$ to P_m
    end for
end for
return $P_0, \dots, P_n$

We shall refer to this improved algorithm as RedZeD.

3 Active enumeration

While the RedZeD algorithm is stated for arbitrary filtrations of chain complexes, our primary motivation is to improve computation times for Vietoris–Rips filtrations. In this setting, RedZeD allows for a natural further improvement, which we call *active enumeration*. In the case of Vietoris–Rips filtrations with active enumeration employed, we call our algorithm RedZeD_VR.

When implemented directly, the persistence pairing algorithm suffers one major bottleneck: when computing n-th persistent homology, almost all time is spent reducing $(n+1)$-birth columns [dSMVJ11,

Bau21]. Since these columns correspond to generators in dimension $n+1$, they are irrelevant to the n-th homology computation, and thus in practice one wants a way to avoid reducing them. In Ripser [Bau21], this is done using a combination of birth clearing and cohomology. These same tricks are not compatible with RedZeD, however, so we require a new method to avoid reducing $(n+1)$-births. We begin by introducing a bit of terminology.

Definition 3.1. Let $X = (X, d)$ be a set equipped with a distance function, and $\mathsf{VR}(X)$ the corresponding (refined) Vietoris–Rips filtration. Let $i \geq 0$, and σ a simplex such that $\mathrm{filt}(\sigma) \leq i$. We say that σ is *active* at time i if $r^i\sigma \neq 0$, otherwise we say σ is *inactive*.

Let us make a few comments about this definition. First, we note that all death simplices are automatically inactive. For birth simplices, the two classifications: alive–vs–dead and active–vs–inactive do not coincide. All inactive simplices are dead and all alive simplices are automatically active, but a simplex can be dead and active, when its homology class was merged with nonzero homology class of another (still alive) simplex. At a given index i, the active simplices in dimension p can be read off from $\mathrm{keys}(R_p)$. Finally, note that all birth simplices start off active, and once they are inactive, they remain inactive.

Let us now explain how the active–vs–inactive distinction could be used to "weed out" some of the $(n+1)$-birth simplices. Proceeding naively, after adding an n-simplex σ, we would look for the next simplices in the filtration, which must be those $(n+1)$-simplices τ that contain σ as their youngest face. As indicated above, we wish to avoid enumerating those that are births. If an $(n+1)$-simplex τ has only inactive faces, then $r\partial\tau = 0$, so τ is an $(n+1)$-birth simplex and we do not need to include it in the filtration. Put differently, we only need to enumerate the $(n+1)$-simplices with at least one active face.

In searching for such $(n+1)$-simplices, no checks are required after a death n-simplex:

Proposition 3.2. *Let (X, d) be a set equipped with a distance function. Suppose a simplex σ of dimension n is added at time i in the (refined) Vietoris–Rips filtration of (X, d) is a death simplex. Then σ does not appear as the youngest face of any $(n+1)$-dimensional simplices.*

Proof. Suppose σ is the youngest face of τ, and τ is the eldest such coface of σ with this property. Then (σ, τ) form an apparent pair in the sense of [Bau21], which contradicts σ being a death simplex. □

Thus, after each birth, simplex σ of dimension n, we search for $(n+1)$-simplices that have at least one active face, and have σ as their youngest face. We do this in two steps:

- **Step 1:** search for all $(n+1)$-simplices that have σ as their youngest face, along with another active face τ; and
- **Step 2:** if σ is still active after the conclusion of Step 1, we search for one extra $(n+1)$-simplex that has σ as its face, and no other active faces.

Step 1: We loop through $\tau \neq \sigma \in \mathrm{keys}(R_n)$ and check for each τ, if there is an $(n+1)$-simplex with σ as its youngest face, that has both σ and τ as faces. To do this, we first check that σ and τ share a face, which happens if $|\sigma \cap \tau| = n$. If so, there is an $(n+1)$-simplex that has σ and τ as faces, but we must ensure that σ is the youngest face of the new simplex. Let $v = \sigma \setminus \tau$ and $w = \tau \setminus \sigma$. We first check if $d(v, w) \leq \mathrm{filt}(\sigma)$, which if so, ensures that $\mathrm{filt}(\sigma \cup \tau) = \mathrm{filt}(\sigma)$, which must be the case for σ to be the youngest face. While this check is not necessary, it is faster than checking directly that σ is the youngest face, so it weeds out cases where we can avoid checking. If $d(v, w) \leq \mathrm{filt}(\sigma)$, we then directly check that σ is the youngest face, and if so, we then process the simplex $\sigma \cup \tau$. To avoid enumerating a simplex twice, we temporarily store a list of the w's that we have checked. Once we have looped through all $\tau \in \mathrm{keys}(R_n)$, we have enumerated all $(n+1)$-simplices that have both σ and at least one other active face in their boundary. If σ is no longer active, then all other $(n+1)$-simplices will have no active faces, so we are done.

Step 2: If σ is still active, we only need to search for one other $(n+1)$-simplex. To do this, we loop through all vertices that are not in the stored list of checked w's, and see if they can be added to σ to form an $(n+1)$-simplex. Once one is found, call it τ, we process it and immediately stop searching for more. This is because since σ is the only active face, we will have that $\overline{\partial}\tau = r\sigma$ (note that we are omitting the index from r here). Thus, when we quotient $\overline{\partial}\tau = 0$, we will get that $r\sigma = 0$, so σ will become inactive. Thus, all future $(n+1)$-simplices created by σ will have no active faces.

Example 3.3. We now revisit Example 2.5 with the addition of active enumeration. Since we are concerned with first homology, when a birth 1-simplex is added, we check for 2-simplices with active faces using step 1 and then step 2. The following table shows how all 2-simplices necessary for the computation of the first homology group are found via active enumeration. The columns contain in order: the 1-simplex x added, the value of $r\partial x$, whether active enumeration applies, the values stored in R_1, and the 2-simplices found in each step.

Simplex x	$r\partial x$	A.E.?	R_1	Step 1 found	Step 2 found
ab	$a+b$	No			
bc	$a+c$	No			
cd	$a+d$	No			
ad	0	Yes	$ad \to ad$		
ac	0	Yes	$ad \to ad$ $ac \to ac$	acd	abc
bd	0	Yes	$bd \to bd$		bcd

Simplices ab, bc, and cd are death simplices, and therefore not subject to active enumeration. When ad is added, since it is a birth, we search for 2-simplices created by ad. Since there are no active simplices that are not ad, we skip step 1. In step 2, no w works to form a 2-simplex, so none are found. After ac is added, we again search for 2-simplices. This time ad is an active face, so we check if $ac \cup ad = acd$ forms a valid 2-simplex created by ac, which it does, so it gets processed. After acd gets processed, we get that $R_1[ac] = R_1[ad] = ad$, so ac is still active. In this case we proced to step 2, where we find abc, which kills ad and makes both ac and ad inactive. When bd gets added, we again skip step 1 since there are no other active 1-simplices. In step 2, we find bcd, which kills bd, and immediately exit the process. In particular, we avoid enumerating abd. Depending on the order we searched, we could have found abd first, and avoided enumerating bcd.

Note that this does not guarantee that all $n+1$ births will not be enumerated, as it is possible for an $(n+1)$-simplex to have multiple active faces and still be a birth. For example, if an $(n+1)$-simplex ω has only active simplices σ and τ, and $r\sigma = r\tau$, then $r\partial\omega = r\sigma + r\tau = 0$. This does, however, avoid enumerating *almost all* $(n+1)$-births. For example, for a dataset of size 400, for $n = 1$ there are generally 10,507,399 dimension 2 birth simplices, and in the worst case we tested, we enumerate 296,911 of them, meaning we avoid enumerating over 97% of all $(n+1)$-dimensional birth simplices. This worst case generally happens when all points are concentrated around one homology feature. For example, this data was taken to be 400 points about a unit circle, with slight noise added in. In other cases, the number is generally much better than 97%. In fact, in all other cases we tested, we avoid enumerating more than 99.95% of the $n+1$ birth simplices, and in some cases, this number is actually 100%. To see the exact types of datasets we tested this on, see Section 5.

Matrix interpretation

Similar to RedZeD, we can translate these ideas into the matrix-theoretic language. Specifically, we translate what it means for a simplex to be active, as well as why simplices with all inactive faces will get reduced to zero.

Recall from the proof of Proposition 2.11 that for a birth simplex σ, if $\sigma \notin R_p[\sigma]$, then $R_p[\sigma]$ is equal to the set of nonzero entries above σ in M^σ, the column with σ as its pivot. If this set is empty, then $R_p[\sigma] = \varnothing$, so we remove it from the dictionary, and σ becomes inactive.

Thus, after the i-th step of the filtration, the active simplices are exactly the simplices that are either current unpaired birth simplices, or birth simplices σ that have been killed, but have nonzero entries above σ in M^σ. Once the column M^σ has been maximally reduced, meaning the only nonzero entry is σ, then σ becomes inactive.

For a matrix interpretation of why $(n+1)$-simplices with inactive faces need not be included in the filtration, suppose an $(n+1)$-simplex τ is such that at the time it is added, all of its faces are inactive simplices. Let M_τ be the column corresponding to τ. Since all faces are inactive, this means that all the faces of τ are either death simplices or ones that appear as a pivot in a column with no other nonzero entries. If this is the case, we know the M_τ will get reduced to zero. This is because any face that is a death simplex will be cleared by the compression algorithm, so that will already be zero in M_τ. For any face σ that is an inactive birth simplex, we can immediately cancel it by adding M^σ to M_τ. Since the rest of M^σ is zero, this does not change the rest of M_τ. Once we do this for all of its faces that are inactive birth simplices, M_τ will be reduced to zero.

Example 3.4. Consider the matrix below, and suppose we have reduced the matrix up until the d-th column.

	a	b	c	d	e
f	0	0	**1**	0	0
~~g~~	~~**1**~~	~~0~~	~~0~~	~~**1**~~	~~**1**~~
h	0	0	**1**	0	0
i	**1**	0	0	**1**	**1**
j	0	**1**	0	**1**	0
k	0	0	0	0	**1**

The crossed out row corresponds to that row being compressed, so it is ignored. The only active simplex that is not alive is h, since it appears as the pivot of column c, which has another nonzero entry in f. The simplices k and f are also active, since they correspond to births not yet killed. We know this because their rows have not been compressed and they are not pivots of any column. Note that i is not active, since the other nonzero entry in column a is zeroed by compression. The column d only has entries in rows i, j, and g. Since g gets compressed the only remaining entries are in i and j, which are both inactive. Thus d would get reduced to zero simply by adding columns a and b. Since d has no active faces, however, the active enumeration shortcut would avoid enumerating and processing column d altogether.

This highlights the fact that the combination of exhaustive and retrospective reduction, along with compression, is necessary for the active enumeration shortcut. Without both exhaustive and retrospective reduction, we wouldn't be reducing all columns as much as possible, and thus wouldn't be able to track active simplices. Without compression, very few simplices would get flagged as inactive. Since death simplices can never be pivots, they are impossible to directly reduce. Thus simplices with at least one death simplex in their boundary may never get flagged as inactive without compression.

4 Implementation

Our implementation of RedZeD_VR was first written in Julia, and then translated to C++, with both versions available at `https://github.com/nkershaw01/RedZeD`. Many parts of the implementation are inspired by Ripser, including the maintenance of neighborhoods for each point, the usage of the

combinatorial numbering system (although we do not make full use of it as in Ripser), and the enclosing radius. Bauer notes in [Bau21] that the enclosing-radius optimization was first proposed by Henselman-Petrusek [HP17].

Input and output. The inputs to RedZeD_VR are a Float32 distance matrix M, and a homological degree n. The matrix can be lower triangular, or square, in which case it will be converted to lower triangular. An optional input is the threshold, in which the filtration is enumerated only up until a specified radius. If no threshold is given, it defaults to the *enclosing radius*, defined as

$$R_e = \min_{x \in X} \max_{y \in X} d(x, y)$$

after which the simplicial complex is a cone, so all homology in non-negative degrees will be trivial. If a threshold is specified, it uses $\min(\text{threshold}, R_e)$. In C++, the output is an array $[P_0, P_1, \ldots, P_n]$ of persistence pairs, where P_i is an array of pairs in degree i. In Julia, the output is a dictionary with keys a dimension d whose value is P_d.

Another optional input is `simplex_pairs`, which defaults to false and, when set to `true`, returns simplex pairs alongside the intervals. This has the same format as the intervals, such that `spx_pairs`$[d][k]$ is the corresponding birth-death simplex pair to the interval `intervals`$[d][k]$. Simplices are stored as vertices, numbered from zero in matrix order. Birth-death pairs are tuples (b, d), where b is the birth simplex and d is the death simplex, with d an empty tuple for infinite intervals.

Filtration enumeration. Rather than enumerate the filtration in strict order, we enumerate and process dimension by dimension, in increasing order. The exception is dimension n, which, due to active enumeration, is processed interleaved with the $(n+1)$-simplices active enumeration finds.

For dimension 1, we build and sort all edges at once, and then process them sequentially. We sort edges via iterated bucket sort, first distributing them into $\min(E/4, 2^{16})$ buckets based on their radius, where E is the total number of edges in the filtration, ignoring the threshold. Both numbers were chosen empirically, with a cap of 2^{16} used for memory, which has the added benefit of speed due to allowing variables such as the counts in each bucket to stay in cache. If a bucket is of size > 32, we again use bucket sort on it, dividing it into (# of edges in the bucket)/4 further buckets. This process is iterated, sorting buckets directly when their size reaches ≤ 32, when the depth reaches 5, or when all edges in a bucket are the same radius. When directly sorting, we use insertion sort when the size ≤ 32, and comparison sort otherwise. Radius ties are broken via the combinatorial index. The function `build_edges` implements this sorting procedure.

The natural approach for dimensions ≥ 2 is to enumerate using the same method; however, this has the drawback of needing to simultaneously store all the simplices in that dimension, which becomes the dominant memory cost. Instead, we generate d-dimensional simplices for $d \geq 2$ one edge at a time: given an edge e, we find all d-dimensional simplices with max edge e using a method similar to Ripser's coface enumeration. For each dimension d, we iterate the edges in order while maintaining a bitset for each vertex v, with 1 in position w if $v \sim w$. Enumerating d-dimensional simplices with max edge e becomes a problem of iteratively intersecting neighbourhoods, which we can do in decreasing vertex order so that generated simplices are done in order of their combinatorial index (which is colexicographical on vertices), which avoids sorting them once they are generated. Simplices are processed immediately after they are found, meaning we do not have to store them. This was observed to be faster, but more importantly, around 16 times more memory efficient for H_2 and higher as opposed to repeating the strategy for edges. This is done in `generate_stream`.

Simplex storage. Once a simplex is generated and processed, it is referenced using its *rank*, its position in the filtration for its dimension, rather than its combinatorial index as in Ripser. The reason for this is

that whenever we calculate $r\partial\sigma \neq 0$, we need to find the youngest element. If we stored simplices via their combinatorial index, this would require decoding the index to vertices, and then performing $d(d+1)/2$ matrix reads to compute the diameter for each element in $r\partial\sigma$. Using rank allows us to choose the largest element in $r\partial\sigma$, which is already sorted (see the paragraph on the symmetric difference). For each live simplex, we also store its diameter, used for reporting persistence pairs. Storing the diameters would avoid the $d(d+1)/2$ matrix reads if the combinatorial index were to be used; however, it would still require accessing memory for each simplex in $r\partial\sigma$ and finding the maximum diameter.

Symmetric difference. To compute symmetric differences, one of the primary operations of the algorithm, we store sets as sorted arrays. To take the symmetric difference of two sets, start at position 1 of each array, advance whichever entry is smaller, and add it to the output list. If at any point the entries are equal, advance both and add neither to the output. When one array is exhausted, append the remaining entries to the output. This method has the added benefit of making pivot selection free, done by simply reading off the last element in the array.

Dictionaries. Since a large portion of the algorithm is spent accessing and editing dictionaries, how they are implemented is important. The main structure we use for the R and R^{-1} dictionaries is `SetDict`, which is built on the structure `FastMap`, an open addressing hash table with linear probing. It uses backwards-shift deletion instead of tombstones, which would otherwise accumulate as simplices are frequently added and deleted as keys, increasing both memory use and running time. The input to `FastMap` is the rank of a simplex, and the output is a pool slot, or 0 if the rank is not a key. The pool slot corresponds to a pool in `SetDict`, which stores arrays `sets`, `sing`, `sdiam`, and `svert`. The array `sets` stores the R-values (or R^{-1} values), `sdiam` stores simplex diameters for R^{-1} (used to return a persistence pair), and `svert` stores simplex vertices for R. The array `sing` is used for R, and stores `sing`[slot] $= 0$ if `sets`[slot] is not a singleton, and `sing`[slot] $= v$ if `sets`[slot] $= \{v\}$. This is used for an active enumeration shortcut, described in the eponymous paragraph.

Since faces arrive as vertex lists and not simplex ranks, we need a way to convert a list of vertices to a pool slot. This is needed for all faces up to dimension $n-1$, as dimension n faces are handled separately by active enumeration. To do this, we use an additional structure `FaceSlot` to map into the R-pool slots. We first convert the vertex set to its combinatorial index and pass it to `FaceSlot`, which uses either a dense or a sparse method to return the R-pool slot. The dense method stores an array `dense`, which is of size the number of possible d-simplices for dimension d, and is such that `dense`$[i]$ is the slot for the simplex with combinatorial index i, or 0 if it is not a key of R_d. The sparse method uses `FastMap` to do the same thing. The reason why a dense method can work here is that this only needs to be stored up to dimension $n-1$, so there will be far fewer potential keys than R, which needs to be used up to dimension n. We switch from dense to sparse when there are more than 2^{24} possible keys, which is mainly useful for large datasets called with a low threshold.

Active enumeration. Recall that active enumeration is done in two steps: step 1 searches for $(n+1)$-simplices τ that have σ as a face and additionally at least one other active face, and step 2 tries to find one additional τ not seen by step 1. Step 1 must first find all active n-simplices that share a face with σ. The most obvious approach is to loop through keys of R_n, and for every key, check if it shares a face with σ. Using this method is slow, however, because there are generally a large number of active simplices and the face-sharing check fails the vast majority of the time. Instead, we store necessary additional information in `ActiveReg`. It stores is a `FaceSlot` table that takes in a combinatorial index of an $(n-1)$-simplex and points at an `ActiveReg` slot. The slot then gives access to the ranks of all active n-simplices containing that face, the extra vertex in the active n-simplex, and its R-pool slot.

Thus, given a birth n-simplex σ, we first compute its faces and use `ActiveReg` to list all active simplices containing each face. For each active τ sharing a face with σ, let w be the extra vertex in τ and v the

extra vertex in σ. We first check that $d(v, w) \leq \text{filt}(\sigma)$ using a similar bitset to that used for filtration enumeration, and if so add w to a list of candidate vertices if it wasn't already present. We also add the slot of τ to a list `seedlists`$[w]$, so that by the end of this process `seedlists`$[w]$ stores the pool slots of all active faces in $\sigma \cup \{w\}$ other than σ itself. This process is done by `collect_candidates`.

After all candidate vertices have been found, we loop through the list to check whether σ is the youngest face of each corresponding coface. Prior to this check, which requires computing the combinatorial index of each face and potentially checking several diameters, we implement the shortcut mentioned in the paragraph on dictionaries. Although active enumeration avoids enumerating most $(n+1)$-births, some are still enumerated. Among the births that do get processed, the vast majority involve cancelling singletons, as when $\sigma \cup \{w\}$ has active faces σ and τ with $R[\sigma] = R[\tau] = \{\omega\}$. Recall that `SetDict` stores an array `sing`, such that `sing`[slot] records whether or not `sets`[slot] is a singleton, and if so stores the single vertex. This allows us to quickly check if a candidate would fall under this case, and if so, immediately skip it. We perform this check before checking if σ is the youngest face of $\sigma \cup \{w\}$, because it is by far the faster check. An alternate shortcut would be to simply compute $r\partial(\sigma \cup \{w\})$ beforehand; however, this this was slower in our experiments. This is because the singleton case catches almost all potential births anyway, and is cheaper because it avoids taking symmetric differences and requires access to an array with a smaller memory footprint.

Step 2 of active enumeration is implemented by `has_leftover_coface`, which intersects the neighbourhoods of all vertices $v \in \sigma$ and removes the vertices seen in step 1. For every remaining vertex w, it checks if σ is the youngest face of $\sigma \cup \{w\}$ and if so returns `true` immediately. If no coface is found, it returns `false`.

5 Experiments

As stated, the primary motivation for the development of RedZeD is to improve computation times and memory usage for Vietoris–Rips persistent homology, for which we now have RedZeD_VR. To our knowledge, the high water mark in this enterprise belongs to the algorithm Ripser [Bau21]. In this section, we compare computation times and memory with Ripser for various datasets.

We compare the C++ version of RedZeD_VR with the C++ implementation of Ripser, version 1.2.1. All experiments were done on a laptop with 64 GB of RAM and an Intel i9-13900HX processor. For a fair comparison, since RedZeD_VR takes input a distance matrix that is also the format we input into Ripser.

We begin by testing on curated datasets, where we can control the characteristics in order to isolate different factors. We also test on some of the common benchmark datasets that are used in [Bau21]. We generate datasets in four ways with different characteristics. The first is `random_euclidean`$(n, \dim)$. This simply generates n random points inside $[0, 1]^{\dim}$. Since these points are random in a cube in $\mathbb{R}^{\dim}$, there is very little topological signal. The second method is `noisy_sphere`$(n, \dim, \sigma)$. This randomly generates n points in $\mathbb{R}^{\dim+1}$ about the unit sphere $S^{\dim}$, and adds noise according to the standard deviation σ. The third method is `grid_spheres`$(n, m, \dim, \sigma)$. This creates n unit spheres $S^{\dim}$ of m points each in $\mathbb{R}^{\dim+1}$, and stacks them in a $\dim+1$-dimensional grid. Again, it adds noise according to the standard deviation σ. Finally, `random_distance`(n) just returns a random $n \times n$ distance matrix, where the diagonal is zero, and it is symmetric. Each entry is chosen at random to be between 0 and 1. While this has no true topological signal, since the dataset is only a semimetric, it creates a large amount of signal due to noise.

We test `random_distance`(n) in homology degrees 1, 2, and 3. For H_1, we test on the range $(50, 450)$, for H_2 we test on the range $(40, 175)$ and for H_3 we use $(30, 125)$. We also test `noisy_sphere`$(n, \dim, \sigma)$ in degrees 1, 2, and 3. For H_1, we set $\dim = 1$ and let n vary in $(200, 3800)$. For H_2 we set $\dim = 2$ and let n vary in $(80, 900)$, and for H_3 we set $\dim = 3$ and let n vary in $(50, 430)$. For `grid_spheres`$(n, m, \dim, \sigma)$ we only test in degrees 1 and 2, since in H_3 we would not be able to increase n to any substantial number. For H_1 we set $m = 20$, $\dim = 1$, and vary n between 5 and 250 (100 and 5000 total points). For H_2 we set

$m = 80$, dim $= 2$, and vary n between 2 and 21 (160 and 1680 total points). For `random_euclidean`$(n, \dim)$, we fix dim $= 2$ and test H_1 for n varying in $(400, 12000)$. We also fix dim $= 10$ and test degrees 1, 2 and 3. For H_1 we let n vary in $(200, 6000)$, for H_2 we use $(80, 1800)$, and for H_3 we use $(40, 330)$. For each experiment, we report the runtime and peak memory usage (peak RSS). The results can be seen below in Fig. 1 and Fig. 2 (note the log scales on both the x and y axes).

Runtime against point count

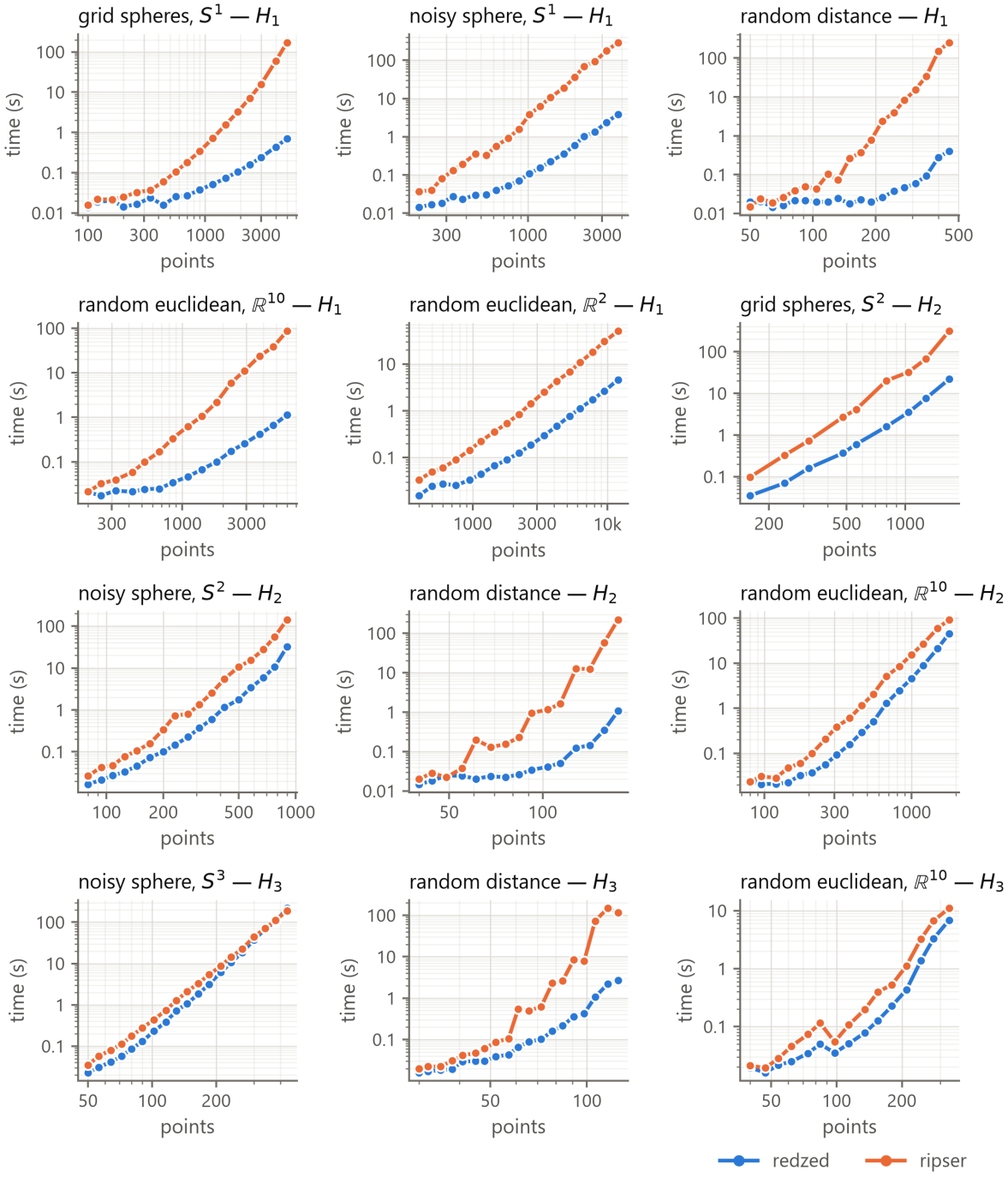


Figure 1: Ripser vs RedZeD_VR time and memory for various datasets

Peak memory against point count

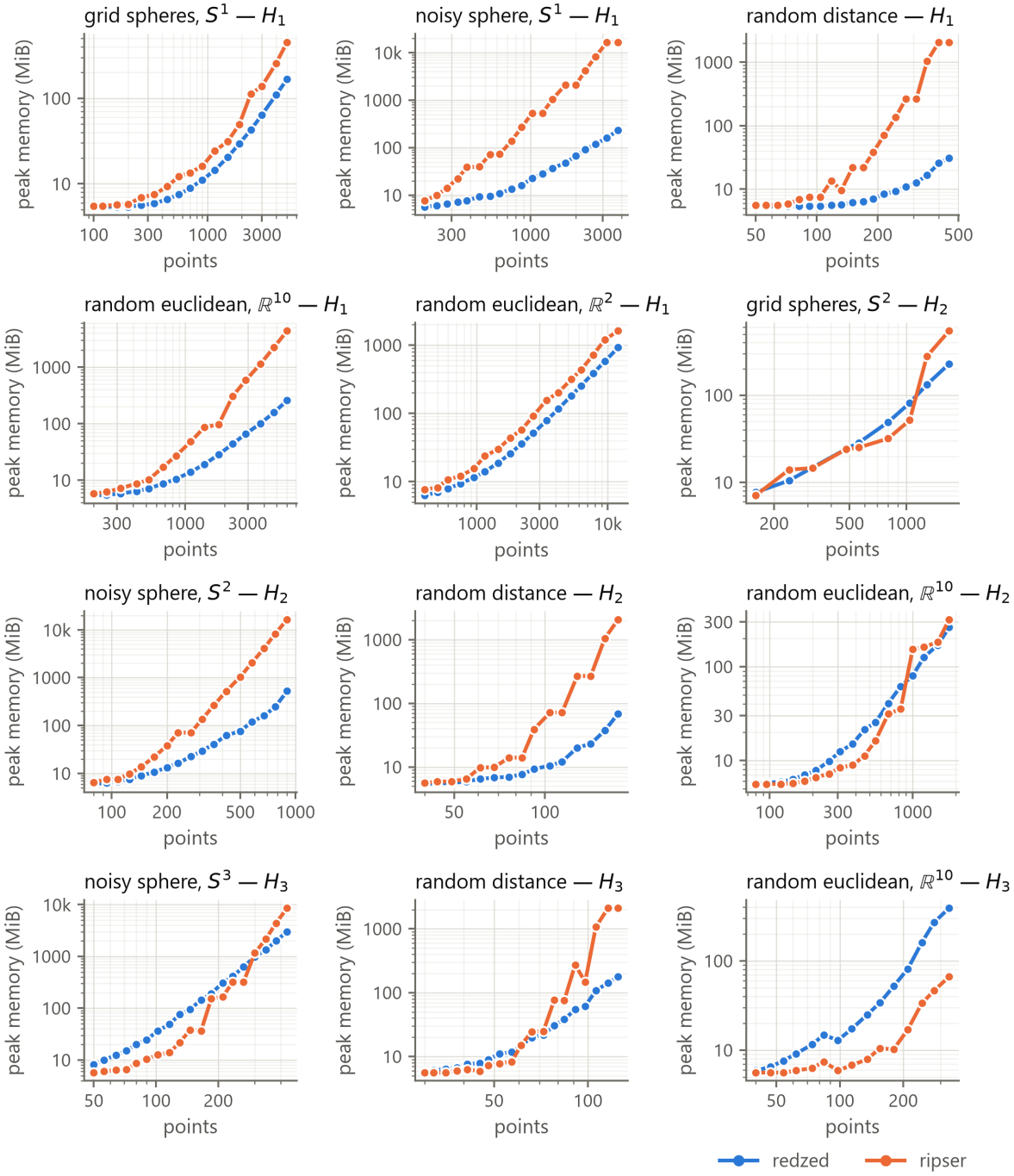


Figure 2: Ripser vs RedZeD_VR time for various datasets

We also run on several of the fixed datasets tested in [Bau21]. Namely, we test on `sphere_3` in degrees

1, 2 and 3, `fract_r` in degrees 1 and 2, `dragon` in degree 1, `o3_1024` in degree 1 and degree 3 with a threshold of 1.8, and finally `o3_4096` in degree 1 and in degree 3 with a threshold of 1.4. The results of those tests, along with the results of the largest runs for the above datasets, can be found below.

			Time (s)			Peak memory (MiB)		
Dataset		pts	RedZeD_VR	Ripser	Speedup	RedZeD_VR	Ripser	Ratio
grid spheres, S^1	H_1	5,000	0.706	171	242	167	452	2.71
noisy sphere, S^1	H_1	3,800	3.87	292	75.4	231	16,500	71.4
random distance	H_1	450	0.404	251	621	30.9	2,070	66.9
random euclidean, $\mathbb{R}^{10}$	H_1	6,000	1.12	86.5	76.9	258	4,400	17.1
random euclidean, $\mathbb{R}^2$	H_1	12,000	4.62	50.7	11.0	921	1,620	1.76
grid spheres, S^2	H_2	1,680	21.9	313	14.3	228	546	2.39
noisy sphere, S^2	H_2	900	32.5	143	4.40	522	16,400	31.4
random distance	H_2	175	1.07	221	207	68.9	2,080	30.3
random euclidean, $\mathbb{R}^{10}$	H_2	1,800	45.4	92.7	2.04	264	319	1.21
noisy sphere, S^3	H_3	430	219	189	0.866	3,000	8,540	2.84
random distance	H_3	125	2.67	116	43.6	178	2,110	11.8
random euclidean, $\mathbb{R}^{10}$	H_3	330	6.87	11.0	1.61	390	66.5	0.171
`sphere3`	H_1	192	0.0167	0.0240	1.43	5.40	6.10	1.13
`sphere3`	H_2	192	0.154	0.441	2.86	16.7	70.2	4.20
`sphere3`	H_3	192	4.24	6.58	1.55	246	86.3	0.351
`fract_r`	H_1	512	0.0235	0.120	5.08	7.50	16.6	2.21
`fract_r`	H_2	512	1.29	3.01	2.34	33.0	18.0	0.545
`o3_1024`	H_1	1,024	0.0599	0.898	15.0	16.4	145	8.82
`o3_1024` (thr. 1.8)	H_3	1,024	0.491	1.36	2.76	81.5	18.2	0.223
`dragon`	H_1	2,000	0.114	0.969	8.51	30.0	98.4	3.28
`o3_4096`	H_1	4,096	1.40	53.3	38.0	177	8,360	47.2
`o3_4096` (thr. 1.4)	H_3	4,096	15.8	26.1	1.65	1,070	179	0.168

Table 2: Time and memory comparisons between RedZeD_VR and Ripser for various datasets

RedZeD_VR is faster than Ripser in all datasets tested except for noisy sphere S^3 for $n = 430$, where it takes 219 seconds compared to Ripser's 189. RedZeD_VR is also more memory efficient in 16/22 of the datasets.

RedZeD_VR appears to perform best in homology degree 1, where it is both faster and more memory efficient in all cases tested. For sufficiently large datasets (ones not affected by startup time), the speedups range from 8.5 to 621 times faster. For speed, the improvements are most significant for high-dimensional data, (76.9 times faster for `random_euclidean`$(6000, 10)$) and data with more topological signal (242, 75.4, and 621 times faster for `grid_spheres`$(250, 20, 1, 0.05)$, `noisy_sphere`$(3800, 1, 0.05)$ and `random_distance`(450) respectively). For memory the biggest improvement is `noisy_sphere`$(3800, 1, 0.05)$, where RedZeD_VR used 321 MiB compared to Ripser's 16,500. The other H_1 datasets ranged from 1.13 to 66.9 times less memory usage.

The H_2 and H_3 numbers are not as pronounced, but RedZeD_VR remains generally faster than Ripser. For H_2 on metric data, speedups range from 2.04 times faster to 14.3, and for the non-metric case RedZeD_VR was 207 times faster. The only loss on memory for H_2 was `fract_r`, where RedZeD_VR was at 33.0 MiB compared to 18.0 MiB for Ripser. For H_3 RedZeD_VR remains faster for all datasets aside from `noisy_sphere`. The two non-thresholded memory losses in H_3 are `sphere3` and `random_euclidean`, the two cases that lack real H_3 signal. For the two H_3 thresholded cases, RedZeD_VR is faster, but loses quite significantly on memory.

RedZeD_VR also appears to scale better than Ripser in many cases, especially in H_1. For `grid_spheres` H_1, at 1500 points RedZeD_VR is 22 times faster than Ripser. At 3000 points this number is 58, and at 5000 points it is 242. In degree 1, we see similar results for time for `noisy_sphere` (17 at $n = 750$, 60 at $n = 2000$, and 75 at $n = 3800$), `random_distance` (15 at $n = 150$, 176 at $n = 275$, and 621 at $n = 450$) and `random_euclidean` $\mathbb{R}^{10}$ (15 for $n = 1400$, 43 for $n = 2900$, and 77 for $n = 6000$). Memory also appears to be scaling better in degree 1 for `noisy_sphere`, `random_euclidean` $\mathbb{R}^{10}$, and `random_distance`.

To highlight the power of active enumeration, we also test RedZeD, the version without active enumeration, on a few datasets:

			Time (s)			Peak memory (MiB)		
Dataset		n	RedZeD_VR	Ripser	RedZeD	RedZeD_VR	Ripser	RedZeD
grid spheres, S^1	H_1	560	0.0256	0.104	4.76	7.50	12.2	2,140
noisy sphere, S^1	H_1	540	0.0306	0.330	7.12	9.60	72.0	3,490
random distance	H_1	450	0.404	251	5.62	30.9	2,070	2,320
random euclidean, $\mathbb{R}^{10}$	H_1	525	0.0243	0.0985	2.89	7.10	10.2	1,620
random euclidean, $\mathbb{R}^2$	H_1	500	0.0246	0.0497	3.76	7.00	8.20	1,750
grid spheres, S^2	H_2	160	0.0352	0.0982	2.39	7.70	7.10	1,080
noisy sphere, S^2	H_2	145	0.0460	0.106	4.29	9.00	13.7	1,930
random distance	H_2	158	0.352	58.1	8.59	37.3	1,050	3,790
random euclidean, $\mathbb{R}^{10}$	H_2	145	0.0224	0.0480	0.263	6.30	5.70	143

Table 3: RedZeD_VR vs Ripser vs RedZeD (no active enumeration)

Interestingly, for `random_distance`, RedZeD still outperforms Ripser; however, it is still significantly outperformed by RedZeD_VR. In all other cases, RedZeD is outperformed by both RedZeD_VR and Ripser by several orders of magnitude. This highlights where the gains come from: the improvements are due to active enumeration, but RedZeD is a necessary step to get there.

6 Conclusion and future directions

In this paper, we introduce two algorithms, RedZeD and RedZeD_VR. In Section 2, we introduce RedZeD, an abstract phrasing and generalization of part of the algorithm introduced by [DFW14]. We also provide a translation between RedZeD and the standard persistence algorithm, noting that it can be viewed as the persistence pairing algorithm with compression, along with exhaustive and retrospective reduction.

We use this abstract phrasing in Section 3, where we discuss *active enumeration*. Our phrasing of the algorithm makes active enumeration a natural further improvement. Namely, the keys in R_n exactly identify the active n-simplices. The benefit of active emueration is that it identifies $(n+1)$-simplices that may contribute to n-th homology, allowing one to ignore almost all birth $(n+1)$-simplices, which are the main bottleneck when RedZeD is implemented directly in this case. In the worst case, we avoid enumerating over 97% of all $(n+1)$-birth simplices, and in general this number is greater than 99%. We call the algorithm with active enumeration implemented RedZeD_VR. The details of our implementation of RedZeD_VR are discussed in Section 4.

Finally, in Section 5, we compare computation times for RedZeD_VR to Ripser [Bau21] in the case of Vietoris–Rips filtrations. In general, RedZeD_VR is both faster and uses less peak memory than Ripser. These results are especially pronounced in degree 1, where speedups range from 8.5 to 621 times faster, and memory improvements range from 1.1 to 71 times less memory used. The improvements are less pronounced, yet still present in higher degrees; however, there are cases where Ripser outperforms RedZeD_VR on either time or memory.

There are several questions and directions that can be considered in future work. While in Section 2, we developed an abstract underpinning of the persistence pairing algorithm with compression, and retrospective/exhaustive reduction, one can also try to view it through the lens of discrete Morse theory [For98, MN13, Sco19].

Another natural direction is to identify other settings where active enumeration might apply. These include discrete homology [BCW14], cubical homology [KMM04], as well as homologies of other filtrations such as Delaunay or Cech filtrations [EM94, BE14]. Finally, it is reasonable to expect that the abstract framing of RedZeD allows for further improvements beyond active enumeration.

References

[Bau21] Ulrich Bauer, *Ripser: efficient computation of Vietoris-Rips persistence barcodes*, J. Appl. Comput. Topol. **5** (2021), no. 3, 391–423, doi:10.1007/s41468-021-00071-5, https://doi.org/10.1007/s41468-021-00071-5.

[BCW14] Hélène Barcelo, Valerio Capraro, and Jacob A. White, *Discrete homology theory for metric spaces*, Bull. Lond. Math. Soc. **46** (2014), no. 5, 889–905.

[BDM15] Jean-Daniel Boissonnat, Tamal K. Dey, and Clément Maria, *The compressed annotation matrix: an efficient data structure for computing persistent cohomology*, Algorithmica **73** (2015), no. 3, 607–619, doi:10.1007/s00453-015-9999-4, https://doi.org/10.1007/s00453-015-9999-4.

[BE14] Ulrich Bauer and Herbert Edelsbrunner, *The Morse theory of čech and Delaunay filtrations*, Computational geometry (SoCG'14), ACM, New York, 2014, pp. 484–490.

[BKR14] Ulrich Bauer, Michael Kerber, and Jan Reininghaus, *Clear and compress: Computing persistent homology in chunks*, Topological Methods in Data Analysis and Visualization III (Cham) (Peer-Timo Bremer, Ingrid Hotz, Valerio Pascucci, and Ronald Peikert, eds.), Springer International Publishing, 2014, pp. 103–117.

[BKRW17] Ulrich Bauer, Michael Kerber, Jan Reininghaus, and Hubert Wagner, *Phat—persistent homology algorithms toolbox*, J. Symbolic Comput. **78** (2017), 76–90, doi:10.1016/j.jsc.2016.03.008, https://doi.org/10.1016/j.jsc.2016.03.008.

[BMG+24] Ulrich Bauer, Talha Bin Masood, Barbara Giunti, Guillaume Houry, Michael Kerber, and Abhishek Rathod, *Keeping it sparse: computing persistent homology revisited*, Comput. Geom. Topol. **3** (2024), no. 1, Art. 6, 26, doi:10.57717/cgt.v3i1.50, https://doi.org/10.57717/cgt.v3i1.50.

[CK11] Chao Chen and Michael Kerber, *Persistent homology computation with a twist*, Proceedings of the 27th European Workshop on Computational Geometry (EuroCG), 2011, pp. 197–200.

[DFW14] Tamal K. Dey, Fengtao Fan, and Yusu Wang, *Computing topological persistence for simplicial maps [extended abstract]*, Computational geometry (SoCG'14), ACM, New York, 2014, pp. 345–354.

[dSMVJ11] Vin de Silva, Dmitriy Morozov, and Mikael Vejdemo-Johansson, *Dualities in persistent (co)homology*, Inverse Problems **27** (2011), no. 12, 124003, 17, doi:10.1088/0266-5611/27/12/124003, https://doi.org/10.1088/0266-5611/27/12/124003.

[ELZ02] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian, *Topological persistence and simplification*, vol. 28, 2002, Discrete and computational geometry and graph drawing (Columbia, SC, 2001), pp. 511–533, doi:10.1007/s00454-002-2885-2, https://doi.org/10.1007/s00454-002-2885-2.

[EM94] Herbert Edelsbrunner and Ernst P. Mücke, *Three-dimensional alpha shapes*, ACM Trans. Graph. **13** (1994), no. 1, 43–72, doi:10.1145/174462.156635, https://doi.org/10.1145/174462.156635.

[EO20] Herbert Edelsbrunner and Katharina Ölsböck, *Tri-partitions and bases of an ordered complex*, Discrete Comput. Geom. **64** (2020), no. 3, 759–775, doi:10.1007/s00454-020-00188-x, https://doi.org/10.1007/s00454-020-00188-x.

[EZ03] Herbert Edelsbrunner and Afra Zomorodian, *Computing linking numbers of a filtration*, vol. 5, 2003, Algebraic topological methods in computer science (Stanford, CA, 2001), pp. 19–37, http://projecteuclid.org/euclid.hha/1088453320.

[For98] Robin Forman, *Morse theory for cell complexes*, Adv. Math. **134** (1998), no. 1, 90–145, doi:10.1006/aima.1997.1650, https://doi.org/10.1006/aima.1997.1650.

[Gab72] Peter Gabriel, *Unzerlegbare Darstellungen. I*, Manuscripta Math. **6** (1972), 71–103; correction, ibid. 6 (1972), 309, doi:10.1007/BF01298413, https://doi.org/10.1007/BF01298413.

[Hir03] Philip S. Hirschhorn, *Model categories and their localizations*, Mathematical Surveys and Monographs, vol. 99, American Mathematical Society, Providence, RI, 2003, doi:10.1090/surv/099, https://doi.org/10.1090/surv/099.

[Hov99] Mark Hovey, *Model categories*, Mathematical Surveys and Monographs, vol. 63, American Mathematical Society, Providence, RI, 1999.

[HP17] Gregory Henselman-Petrusek, *Matroids and canonical forms: Theory and applications*, Ph.D. thesis, University of Pennsylvania, 2017, arXiv:1710.06084.

[KK25] Chris Kapulkin and Nathan Kershaw, *Data analysis using discrete cubical homology*, 2025, arXiv:2506.15020, doi:10.48550/arXiv.2506.15020, https://arxiv.org/abs/2506.15020.

[KMM04] Tomasz Kaczynski, Konstantin Mischaikow, and Marian Mrozek, *Cubical homology*, pp. 39–92, Springer New York, New York, NY, 2004, doi:10.1007/0-387-21597-2_2, https://doi.org/10.1007/0-387-21597-2_2.

[MN13] Konstantin Mischaikow and Vidit Nanda, *Morse theory for filtrations and efficient computation of persistent homology*, Discrete Comput. Geom. **50** (2013), no. 2, 330–353, doi:10.1007/s00454-013-9529-6, https://doi.org/10.1007/s00454-013-9529-6.

[OPT+17] Nina Otter, Mason A. Porter, Ulrike Tillmann, Peter Grindrod, and Heather A. Harrington, *A roadmap for the computation of persistent homology*, EPJ Data Science **6** (2017), no. 1, 17, doi:10.1140/epjds/s13688-017-0109-5, https://doi.org/10.1140/epjds/s13688-017-0109-5.

[Sco19] Nicholas A. Scoville, *Discrete Morse theory*, Student Mathematical Library, vol. 90, American Mathematical Society, Providence, RI, 2019, doi:10.1090/stml/090, https://doi.org/10.1090/stml/090.

[ZC05] Afra Zomorodian and Gunnar Carlsson, *Computing persistent homology*, Discrete Comput. Geom. **33** (2005), no. 2, 249–274, doi:10.1007/s00454-004-1146-y, https://doi.org/10.1007/s00454-004-1146-y.